

Clowns to the Left of me, Jokers to the Right



A collection of *functional pearls*
on algebraic data structures.

There's an old *Stealer's Wheel* song with the memorable chorus:

*'Clowns to the left of me, jokers to the right,
Here I am, stuck in the middle with you.'*

Joe Egan, Gerry Rafferty

In this paper, I examine what it's like to be stuck in the middle of traversing and transforming a data structure.



Zipper

A zipper inserts an n hole context in a data structure.

Zipper

A zipper has n pointers focussing on specific elements in a data structure.

Zipper

A zipper has n pointers focussing on specific elements in a data structure.

Take any data structure, and walk to any element in it.

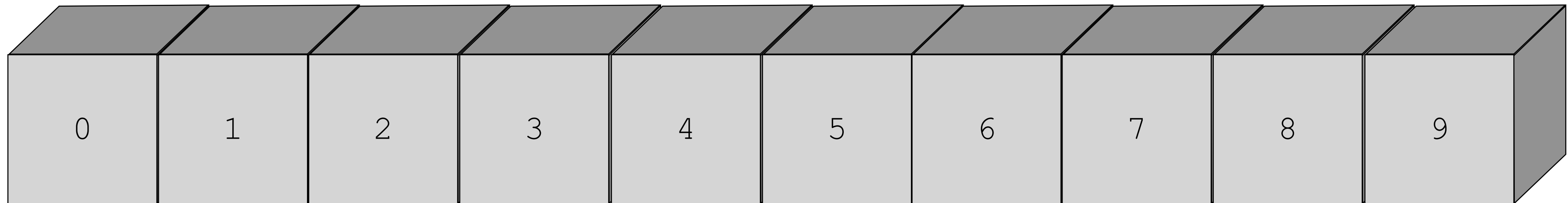
What do you see?

List Zipper

Consider the list `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

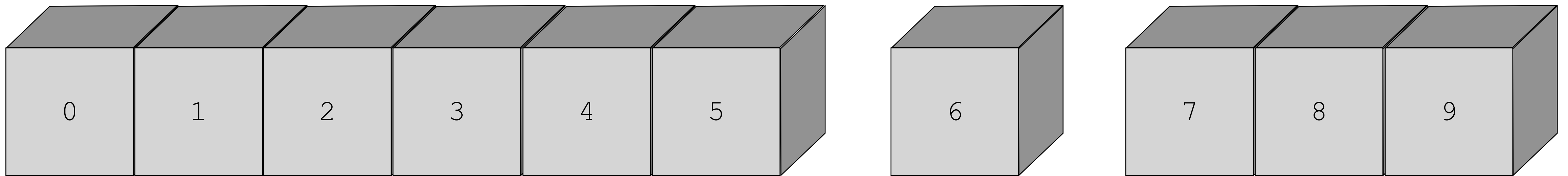
List Zipper

Consider the list `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`



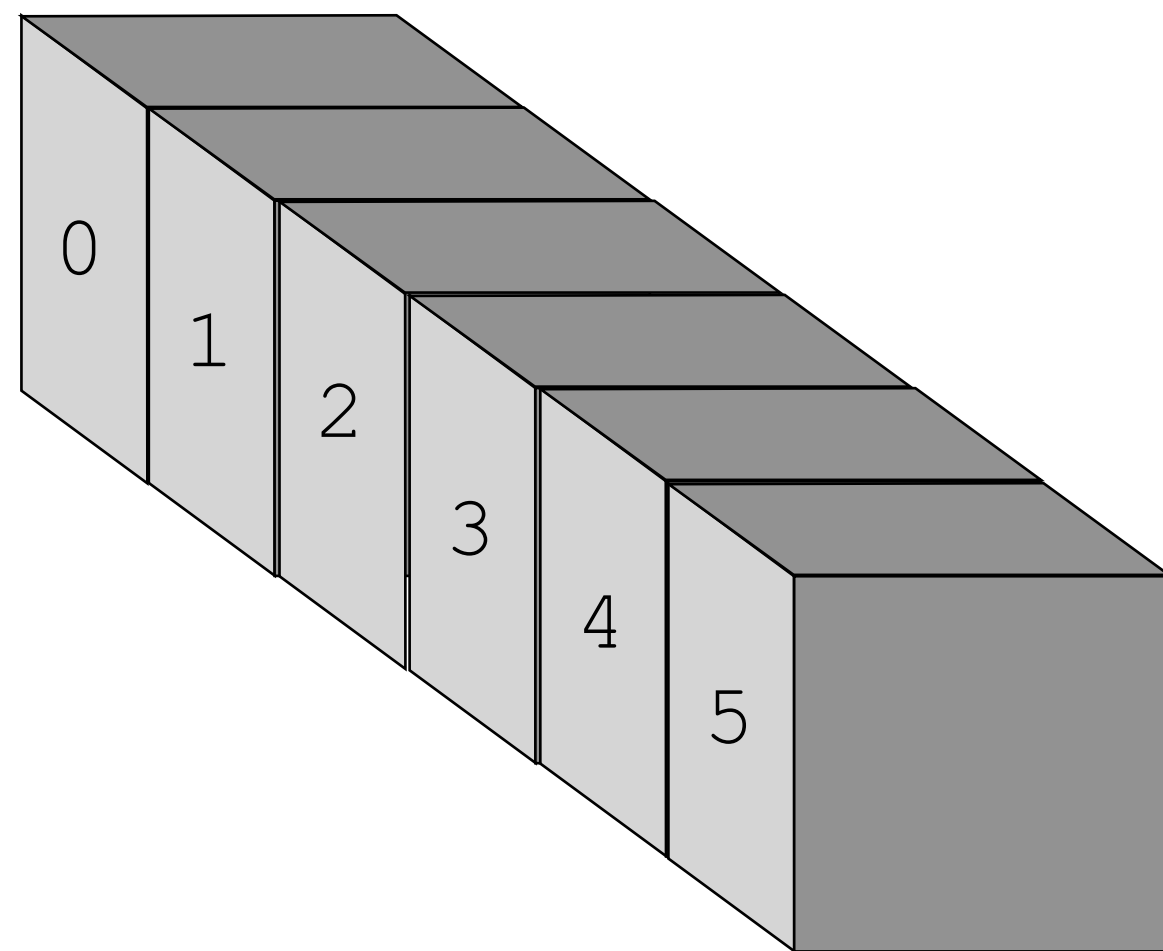
List Zipper

Consider the list `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

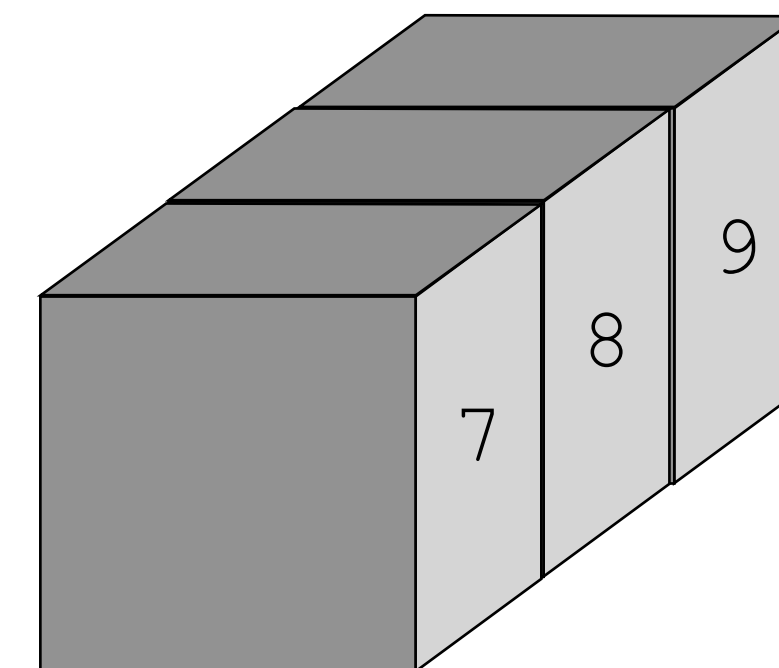
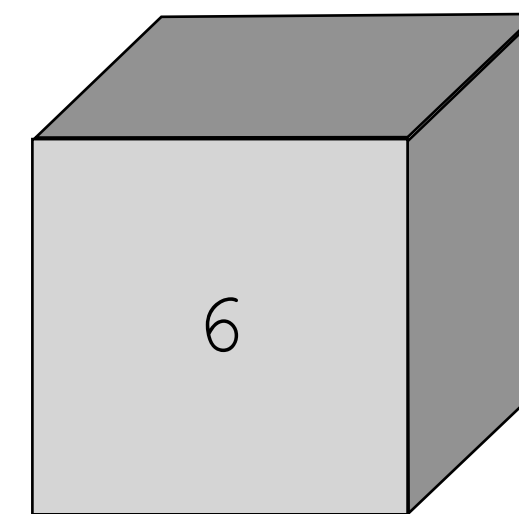


List Zipper

Consider the list `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`



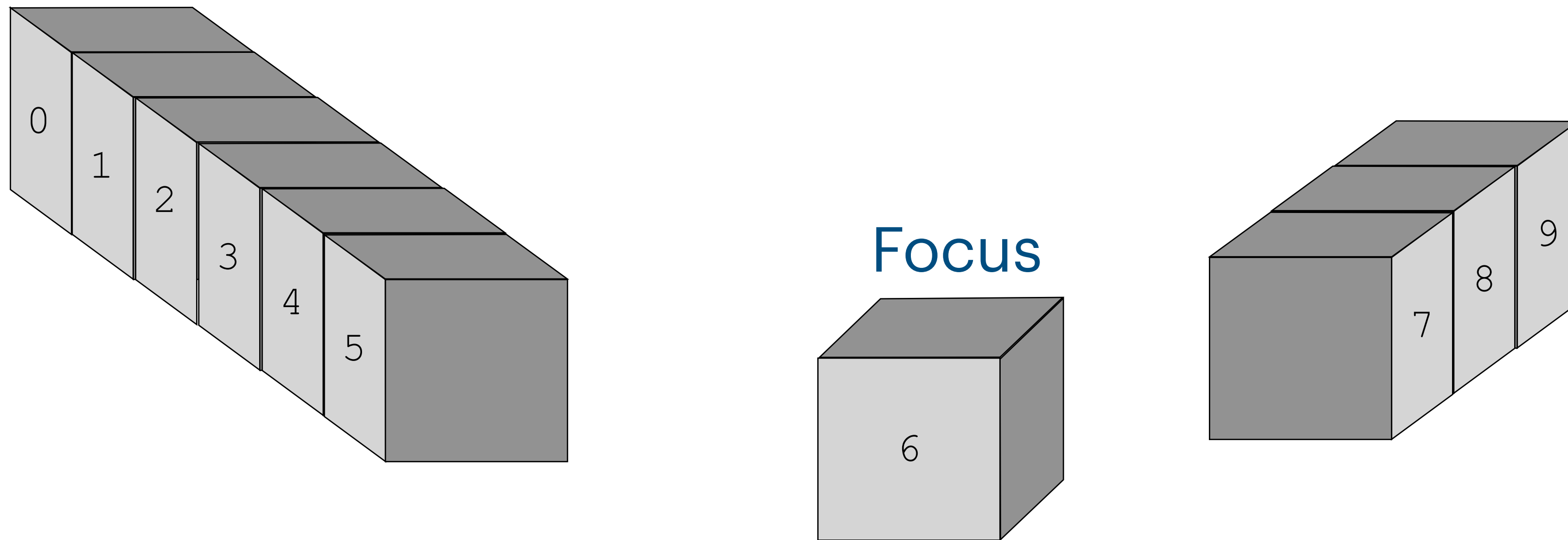
Just look around
yourself!



`([5, 4, 3, 2, 1, 0], 6, [7, 8, 9])`

List Zipper

Consider the list `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`



`([5, 4, 3, 2, 1, 0], 6, [7, 8, 9])`

List Zipper

```
listz =  
    ([5, 4, 3, 2, 1, 0], 6, [7, 8, 9])
```

```
moveLeft listz =  
    ([4, 3, 2, 1, 0], 5, [6, 7, 8, 9])
```

$\Theta(1)$



List Zipper

The zipper for $[a]$ is $([a], a, [a])$.



List Zipper

```
data ListZipper a =  
  ListZipper [a] a [a]
```



List Zipper

```
data ListZipper a =  
    ListZipper [a] a [a]
```

```
moveLeft    :: ListZipper a -> Maybe (ListZipper a)
```

```
moveRight   :: ListZipper a -> Maybe (ListZipper a)
```



List Zipper

```
data ListZipper a =  
    ListZipper [a] a [a]
```

```
moveLeft    :: ListZipper a -> Maybe (ListZipper a)
```

```
moveRight   :: ListZipper a -> Maybe (ListZipper a)
```

```
findLeft    :: (a -> Bool) -> ListZipper a ->  
                                                     Maybe (ListZipper a)
```

```
modify      :: (a -> a) -> ListZipper a -> ListZipper a
```

List Zipper

```
data ListZipper a =  
  ListZipper [a] a [a]
```

```
delete :: ListZipper a -> Maybe (ListZipper a)
```

Delete the focus I am on.

List Zipper

```
data ListZipper a =  
    ListZipper [a] a [a]
```

```
deleteLeft  :: ListZipper a -> Maybe (ListZipper a)
```

```
deleteRight :: ListZipper a -> Maybe (ListZipper a)
```

Delete the focus I am on.

deleteLeft **or** deleteRight



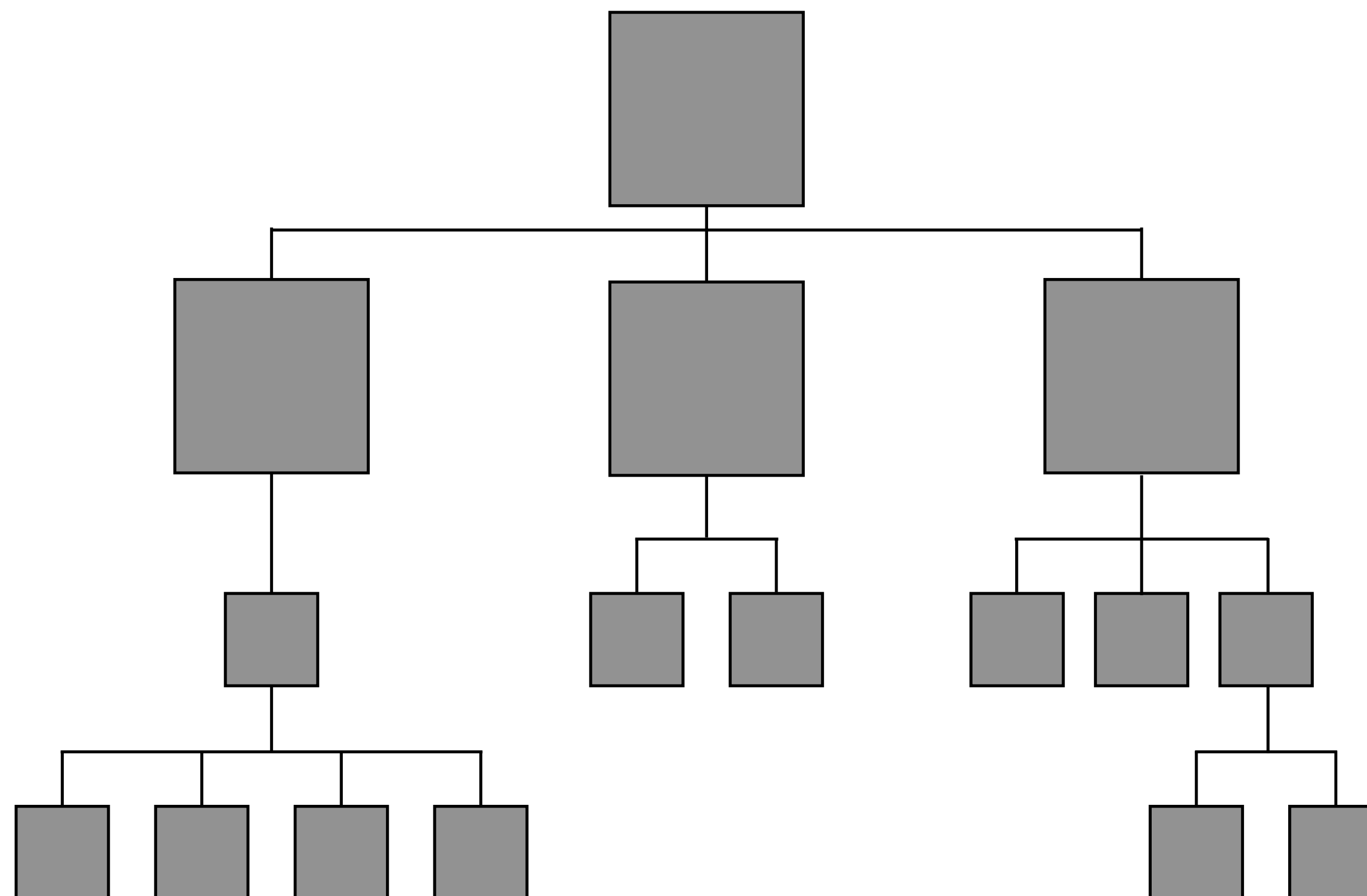
What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```



What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```

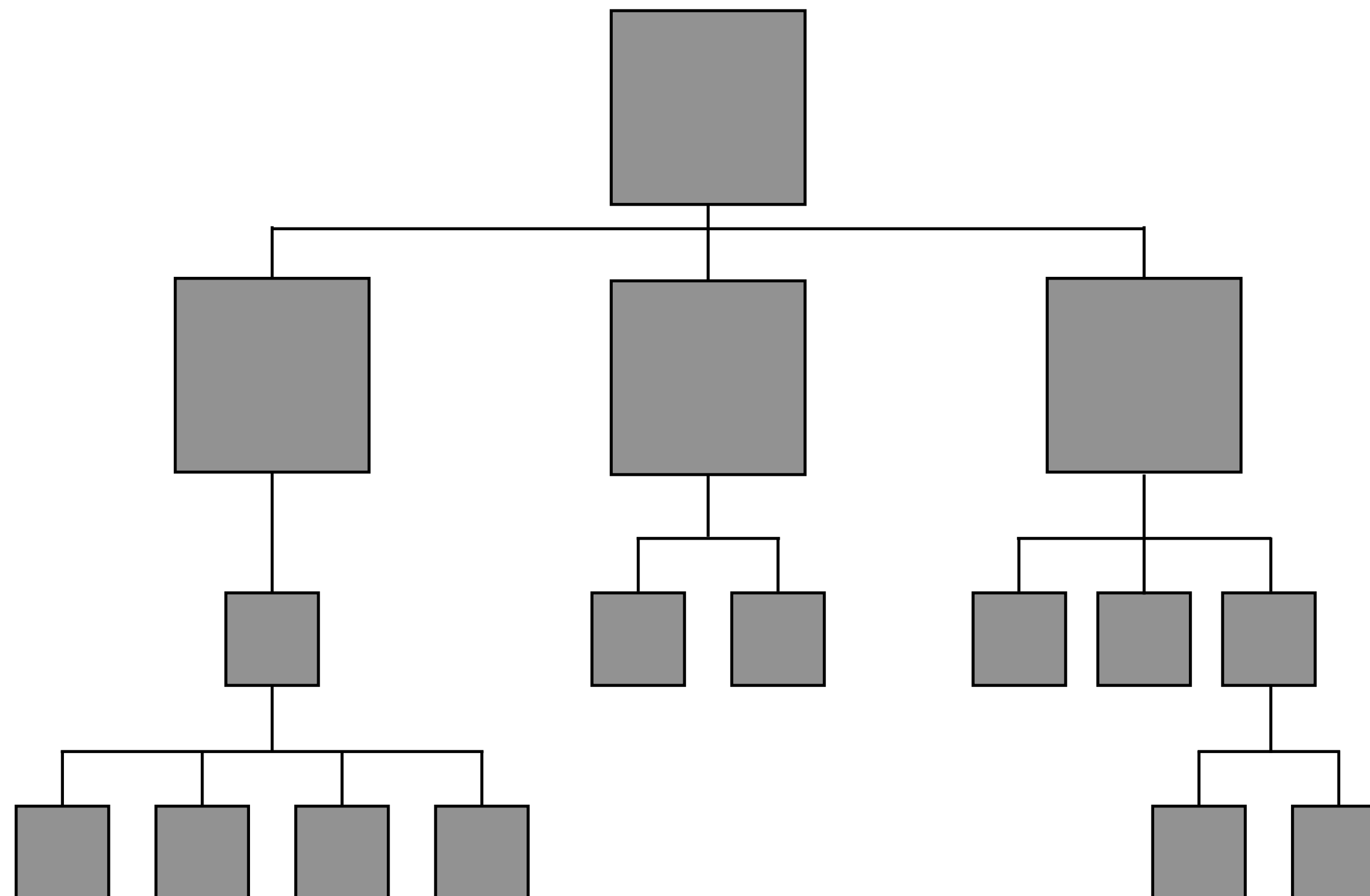




What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```

```
leftSiblings :: [Tree a]
```

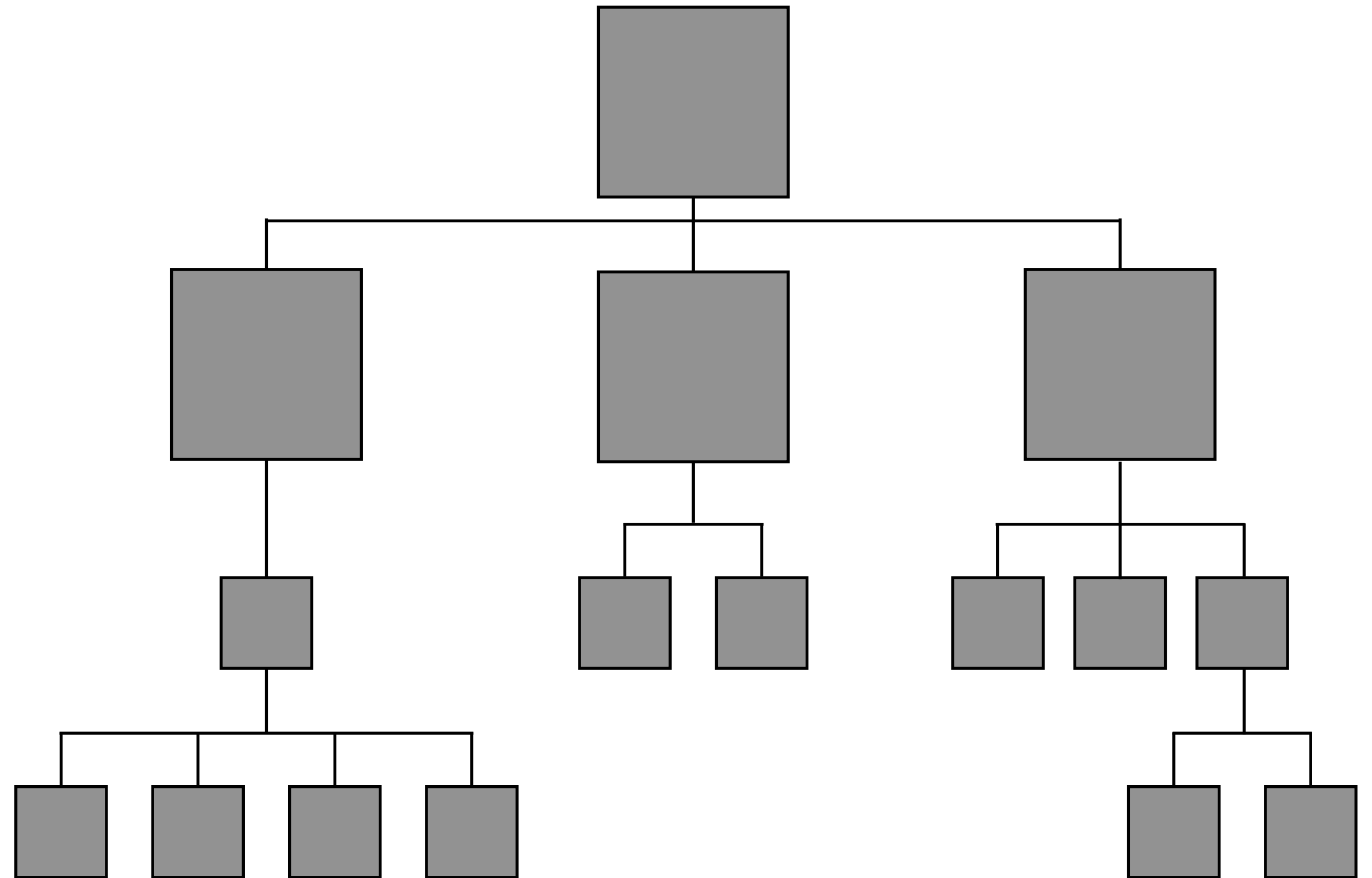




What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```

```
leftSiblings  :: [Tree a]  
rightSiblings :: [Tree a]
```





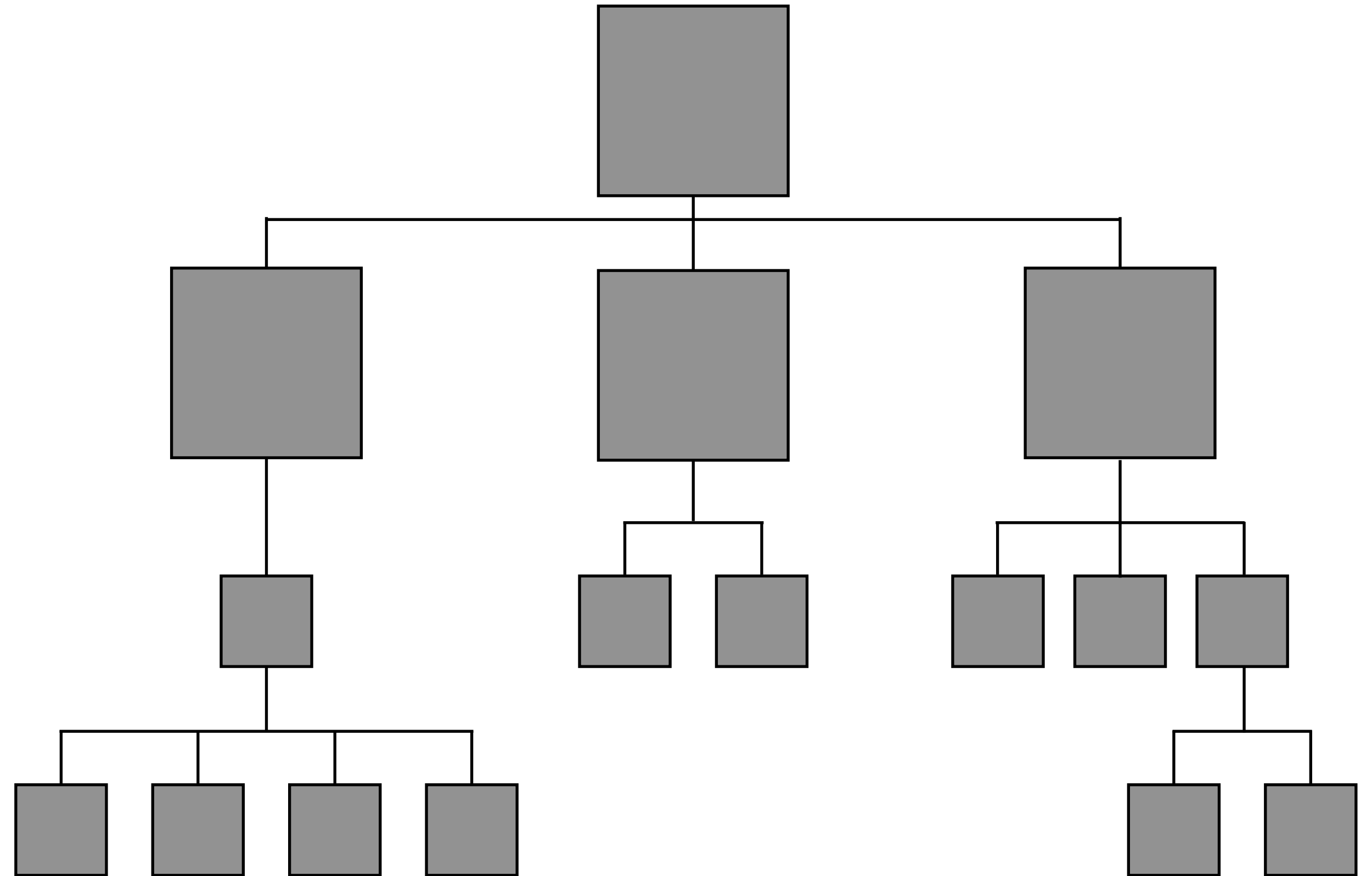
What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```

```
leftSiblings  :: [Tree a]
```

```
rightSiblings :: [Tree a]
```

```
focus      :: a
```





What about Multiway Trees?

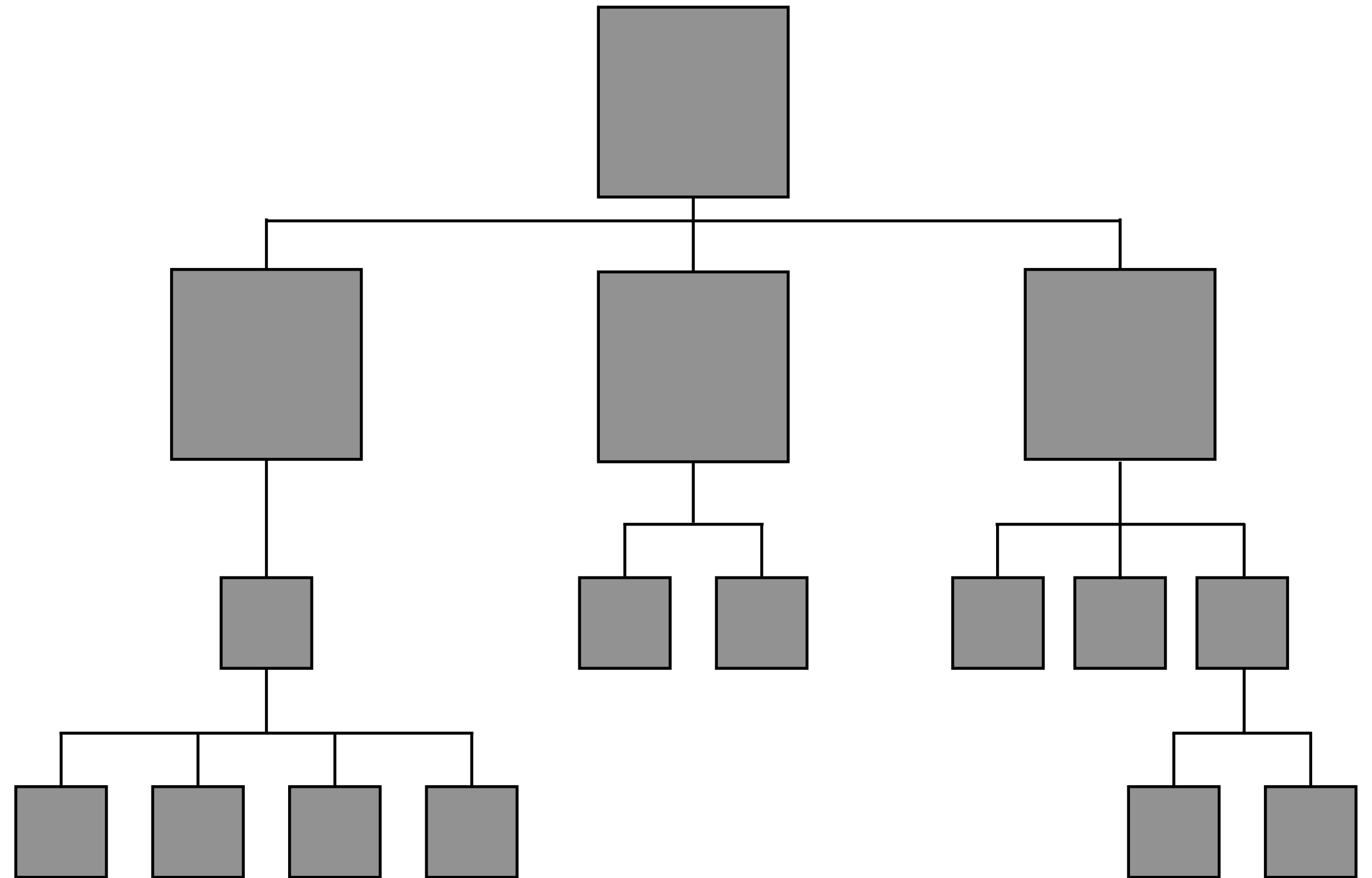
```
data Tree a =  
  Tree a [Tree a]
```

```
leftSiblings  :: [Tree a]
```

```
rightSiblings :: [Tree a]
```

```
focus      :: a
```

```
children    :: [Tree a]
```





What about Multiway Trees?

```
data Tree a =  
  Tree a [Tree a]
```

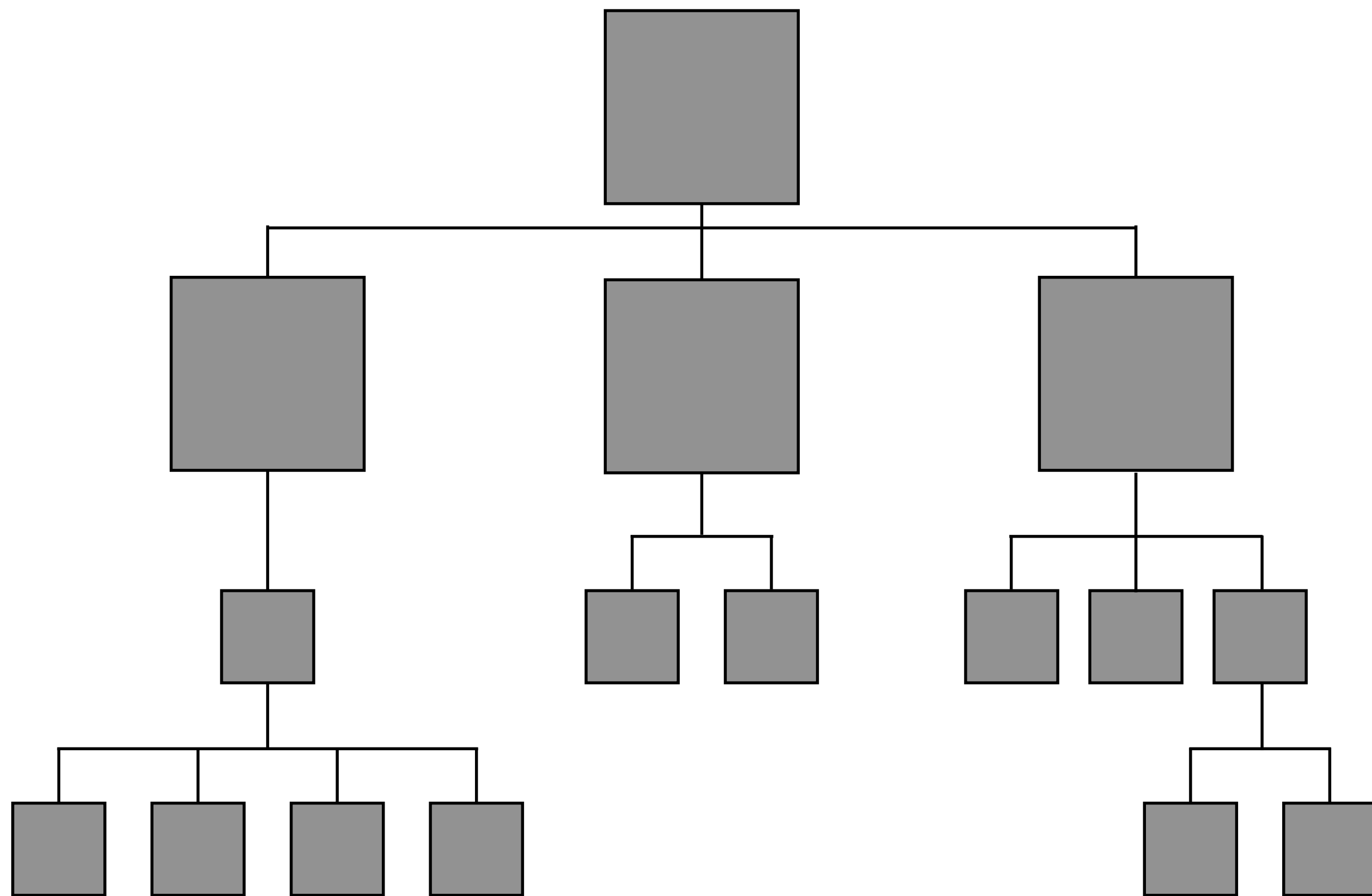
```
leftSiblings  :: [Tree a]
```

```
rightSiblings :: [Tree a]
```

```
focus       :: a
```

```
children     :: [Tree a]
```

```
parent       :: ??
```



FUNCTIONAL PEARL

The Zipper

GÉRARD HUET

INRIA Rocquencourt, France

Capsule Review

Almost every programmer has faced the problem of representing a tree together with a subtree that is the focus of attention, where that focus may move left, right, up or down the tree. The Zipper is Huet's nifty name for a nifty data structure which fulfills this need. I wish I had known of it when I faced this task, because the solution I came up with was not quite so efficient or elegant as the Zipper.

1 Introduction

The main drawback to the purely applicative paradigm of programming is that many efficient algorithms use destructive operations in data structures such as bit vectors or character arrays or other mutable hierarchical classification structures, which are not immediately modelled as purely applicative data structures. A well

Types as Arithmetic

Type	Values	Count
<code>Void</code>	<code>none</code>	0
<code>()</code>	<code>()</code>	1
<code>Bool</code>	<code>True, False</code>	2
<code>Maybe Bool</code>	<code>Nothing, Just True, Just False</code>	3
<code>(Bool, Bool)</code>	<code>(True, True), (False, True), (True, False), (False, False)</code>	4
<code>Maybe (Bool, Bool)</code>	<code>Nothing, Just ...</code>	5

Every finite type has a size.

Each natural number n can be associated with a type.

Type	Values	Count
<code>Void</code>	<code>none</code>	0
<code>()</code>	<code>()</code>	1
<code>Bool</code>	<code>True, False</code>	2
<code>Maybe Bool</code>	<code>Nothing, Just True, Just False</code>	3
<code>(Bool, Bool)</code>	<code>(True, True), (False, True), (True, False), (False, False)</code>	4
<code>Maybe (Bool, Bool)</code>	<code>Nothing, Just ...</code>	5

Product Types

```
data Pair a b = Pair a b
```

A value of `Pair a b` is a value
of `a` and a value of `b`.

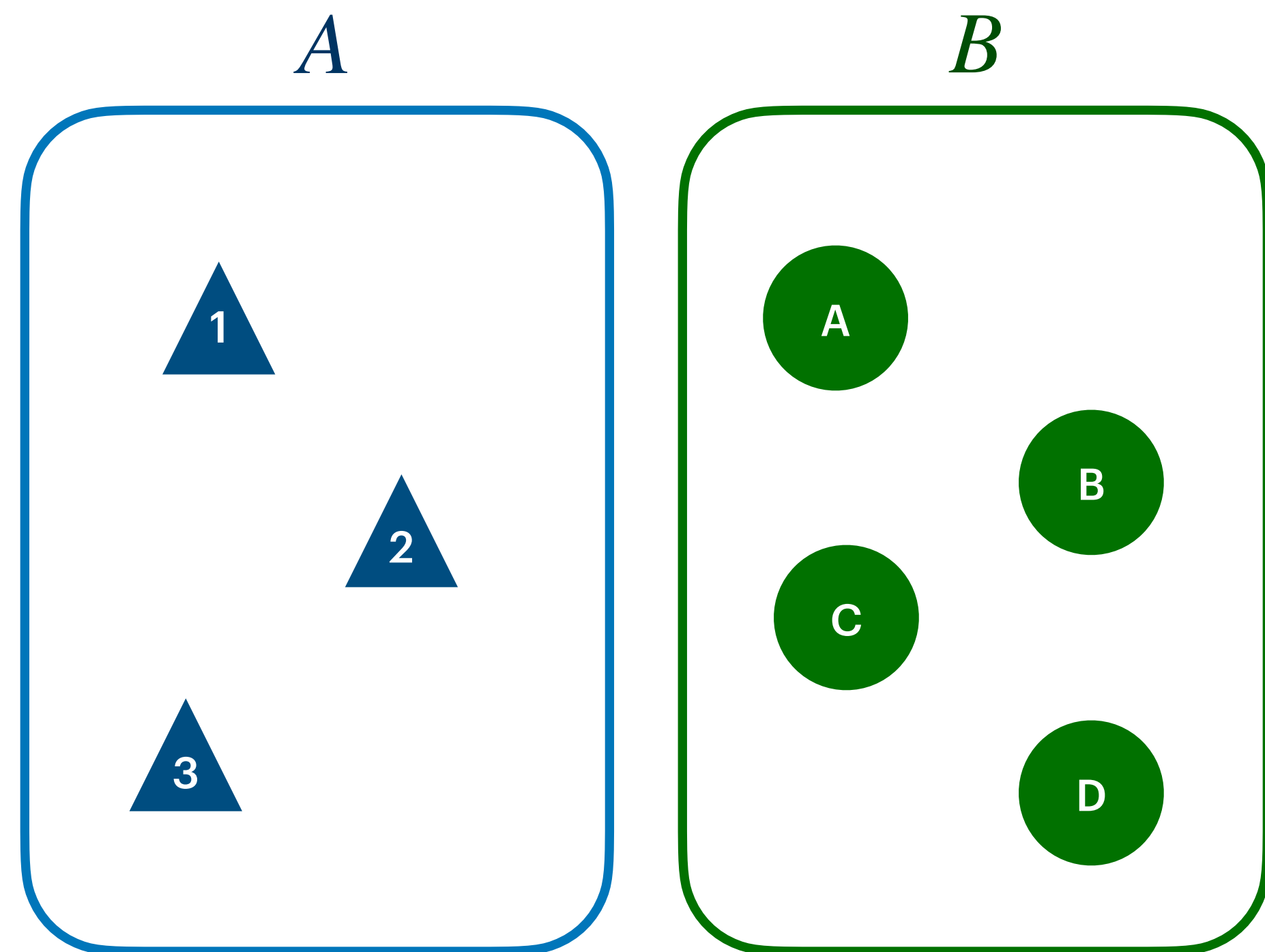
Product Types

```
data Pair a b = Pair a b
```

A value of `Pair a b` is a value of `a` and a value of `b`.

`(a, b)`

$A \times B$



$$|A \times B| = |A| \times |B| = 12$$

Sum Types

```
data Either a b = Left a | Right b
```

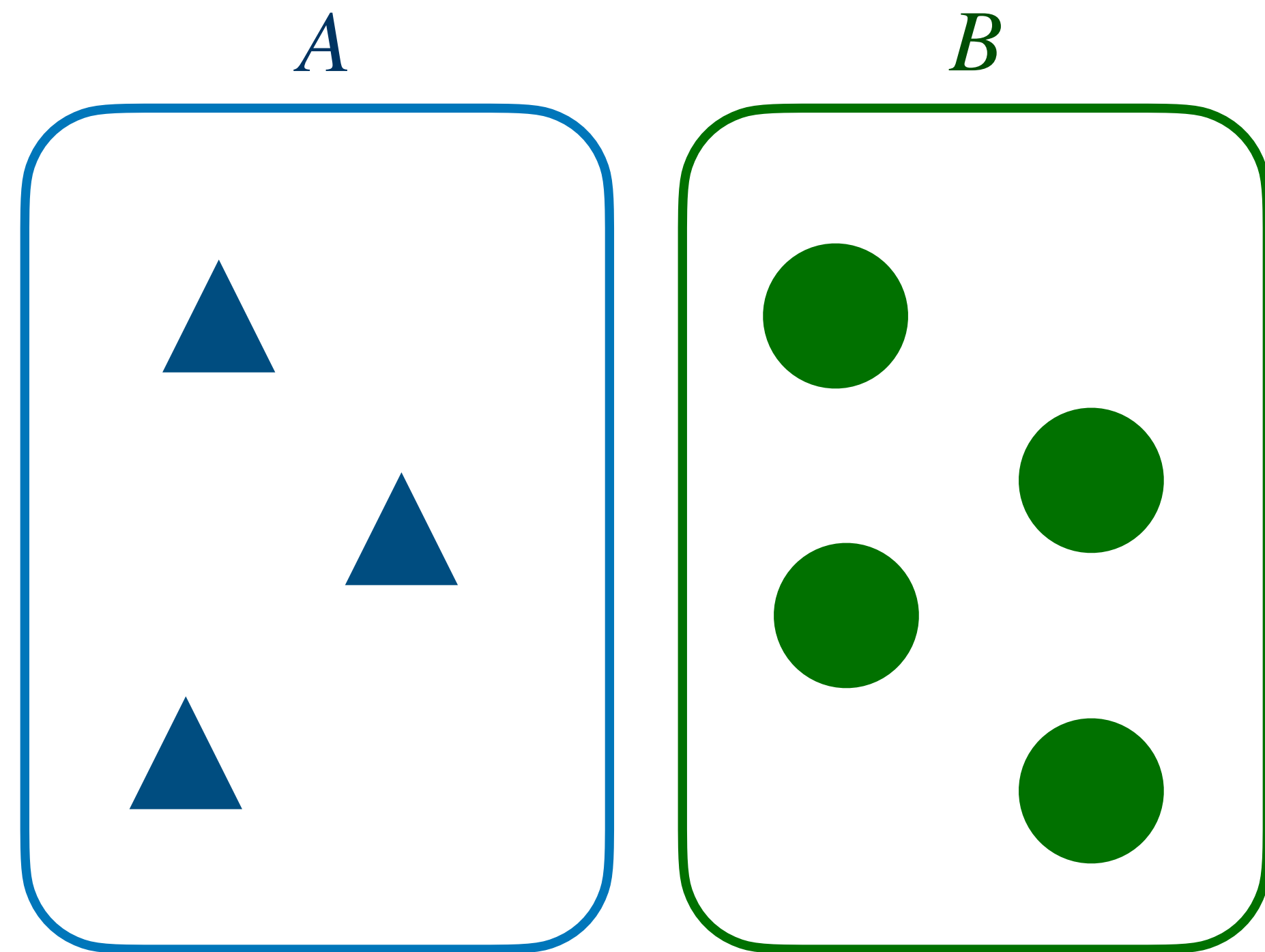
A value of `Either a b` is a
value of `a` or a value of `b`,
tagged so we know which.

Sum Types

```
data Either a b = Left a | Right b
```

A value of `Either a b` is a value of `a` or a value of `b`, tagged so we know which.

$\text{bool} \cong 1 + 1$



$$|A + B| = |A| + |B| = 7$$

Types as Arithmetic

Exercises:

1. Prove that `Pair` satisfies the universal property of products.
2. Prove that `Either` satisfies the universal property of co-products.
3. Prove that $\text{Maybe}(A) \cong 1 + A$.
4. Show that algebraic types form a *commutative semi-ring*:

$$A + B \cong B + A, \quad (A + B) + C \cong A + (B + C), \quad A + 0 \cong A.$$

$$A \times B \cong B \times A, \quad (A \times B) \times C \cong A \times (B \times C), \quad A \times 1 \cong A.$$

$$A \times (B + C) \cong A \times B + A \times C. \quad A \times 0 \cong 0$$

Types as Arithmetic

`distr :: (a, Either b c) -> Either (a, b) (a, c)`

`distr (a, Left b) = Left (a, b)`

`distr (a, Right c) = Right (a, c)`

`undistr :: Either (a, b) (a, c) -> (a, Either b c)`

`undistr (Left (a, b)) = (a, Left b)`

`undistr (Right (a, c)) = (a, Right c)`

The laws of arithmetic hold, as programs.

Types as Polynomials

Haskell	polynomial	reading
<code>Maybe a</code>	$1 + X$	nothing, or one element
<code>(a, a)</code>	X^2	two elements
<code>(a, a, a)</code>	X^3	three elements
<code>(Bool, a)</code>	$2X$	a tag and one element
<code>Either a (a, a)</code>	$X + X^2$	one element, or two

A term cX^n means: **c** different shapes with **n** slots for elements.

Recursive Types

```
data [a] = [] | a : [a]
```

$$L(A) = 1 + A \times L(A)$$

Recursive Types

```
data [a] = [] | a : [a]
```

$$L(A) = 1 + A \times L(A)$$

```
data Tree a = Tree a [Tree a]
```

Recursive Types

```
data [a] = [] | a : [a]
```

$$L(A) = 1 + A \times L(A)$$

```
data Tree a = Tree a [Tree a]
```

$$T(A) = A \times L(T(A))$$

Recursive Types

```
data [a] = [] | a : [a]
```

$$L(A) = 1 + A \times L(A)$$

```
data Tree a = Tree a [Tree a]
```

$$T(A) = A \times L(T(A))$$

A recursive type is a solution of its equation.

Recursive Types

$$L = 1 + A \times L$$

Recursive Types

$$L = 1 + A \times L$$

$$= 1 + A \times (1 + A \times L) = 1 + A + A^2 \times L$$

Recursive Types

$$L = 1 + A \times L$$

$$= 1 + A \times (1 + A \times L) = 1 + A + A^2 \times L$$

$$= 1 + A + A^2 + A^3 \times L$$

Recursive Types

$$L = 1 + A \times L$$

$$= 1 + A \times (1 + A \times L) = 1 + A + A^2 \times L$$

$$= 1 + A + A^2 + A^3 \times L$$

$$= 1 + A + A^2 + A^3 + \dots$$

Unfolding forever gives the infinite lists.

Recursive Types

Type	polynomial
Natural Numbers	$T = 1 + T$
Lists	$T = 1 + X \times T$
Binary Trees	$T = 1 + X \times T^2$
Multiway Trees	$T = X \times L(T)$

What if we differentiate the polynomial?

Differentiating Algebraic Types

$$L = 1 + A \times L$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$\frac{\partial L}{\partial A} = \frac{\partial 1}{\partial A} + \frac{\partial}{\partial A} (A \times L)$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$\frac{\partial L}{\partial A} = \frac{\partial}{\partial A} (A \times L)$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$\frac{\partial L}{\partial A} = \frac{\partial A}{\partial A} \times L + A \times \frac{\partial L}{\partial A}$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$\frac{\partial L}{\partial A} = L + A \times \frac{\partial L}{\partial A}$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

```
data Z a = Stop [a] | Step a (Z a)
```

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

```
data Z a = Stop [a] | Step a (Z a)
```

Claim: $\text{data } Z \ a \cong ([a], [a])$

Differentiating Algebraic Types

```
data Z a = Stop [a] | Step a (Z a)
```

Claim: $\text{data } Z \ a \cong ([a], [a])$

```
to :: Z a -> ([a], [a])
to (Stop rs)      = ([], rs)
to (Step x z)     = (x : ls, rs)
  where (ls, rs) = to z
```

```
from :: ([a], [a]) -> Z a
from ([], rs)      = Stop rs
from (x : ls, rs) = Step x (from (ls, rs))
```

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

$$= L + A \times (L + A \times Z)$$

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

$$= L + A \times L + A^2 \times Z$$

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

$$= L + A \times L + A^2 \times Z$$

$$= L + A \times L + A^2 \times L + A^3 \times L + \dots$$

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

$$= L + A \times L + A^2 \times Z$$

$$= L + A \times L + A^2 \times L + A^3 \times L + \dots$$

$$= (1 + A + A^2 + A^3 + \dots) \times L$$

Differentiating Algebraic Types

Method 2:

$$L = 1 + A \times L$$

$$Z = L + A \times Z$$

$$= L + A \times L + A^2 \times Z$$

$$= L + A \times L + A^2 \times L + A^3 \times L + \dots$$

$$= L \times L$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

$$\partial L = \partial(1 - A)^{-1}$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

$$\partial L = (1 - A)^{-2} \cdot \partial A$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

$$\partial L = L^2 \cdot \partial A$$

Differentiating Algebraic Types

Method 3:

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

$$\partial L = L^2 \cdot \partial A$$

$$Z = L^2$$

Differentiating Algebraic Types

$$L = 1 + A \times L$$

$$L = (1 - A)^{-1}$$

$$\partial L = L^2 \cdot \partial A$$

$$Z = L^2$$

But what is ∂L ?

Differentiating Algebraic Types

From calculus: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$

Differentiating Algebraic Types

From calculus: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$

Types	Meaning
<code>[Either a h]</code>	lists of elements or markers
<code>drop [a]</code>	keep lists with at least one marker
split at the first marker	remove one marker
<code>h = Void</code>	no further markers

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

Types	Meaning
<code>[Either a h]</code>	lists of elements or markers
<code>drop [a]</code>	keep lists with at least one marker
split at the first marker	remove one marker
<code>h = Void</code>	no further markers

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

```
split :: [Either a h] -> Either [a] ([a], h, [Either a h])
```

```
split [] = Left []
```

```
split (Right h : xs) = Right ([], h, xs)
```

```
split (Left x : xs) =
```

```
  case split xs of
```

```
    Left as -> Left (x : as)
```

```
    Right (ps, h, rest) -> Right (x : ps, h, rest)
```

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

```
split :: [Either a h] -> Either [a] ([a], h, [Either a h])
```

```
split [] = Left []
```

```
split (Right h : xs) = Right ([], h, xs)
```

```
split (Left x : xs) =
```

```
  case split xs of
```

```
    Left as -> Left (x : as)
```

```
    Right (ps, h, rest) -> Right (x : ps, h, rest)
```

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

```
split :: [Either a h] -> Either [a] ([a], h, [Either a h])
```

```
split [] = Left []
```

```
split (Right h : xs) = Right ([], h, xs)
```

```
split (Left x : xs) =
```

```
  case split xs of
```

```
    Left as -> Left (x : as)
```

```
    Right (ps, h, rest) -> Right (x : ps, h, rest)
```

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

```
split :: [Either a h] -> Either [a] ([a], h, [Either a h])
```

```
split [] = Left []
```

```
split (Right h : xs) = Right ([], h, xs)
```

```
split (Left x : xs) =
```

```
  case split xs of
```

```
    Left as -> Left (x : as)
```

```
    Right (ps, h, rest) -> Right (x : ps, h, rest)
```

Differentiating Algebraic Types

Every list with a marker splits uniquely at its first marker:

$$L(X + H) \cong L(X) + \underbrace{H \times L(X) \times L(X + H)}_{Q(X,H)}$$

Now let $h \rightarrow 0$: put `h = Void`.

$$Q(X,0) = L(X) \times L(X)$$

Differentiating Algebraic Types

[0, 1, 2, 3, 4, 5, *, 7, 8, 9]

Differentiating Algebraic Types

`[0, 1, 2, 3, 4, 5, *, 7, 8, 9]`

split at the
first marker

`( [0, 1, 2, 3, 4, 5], * , [7, 8, 9] )`

Differentiating Algebraic Types

`[0, 1, 2, 3, 4, 5, *, 7, 8, 9]`

split at the
first marker

`([0, 1, 2, 3, 4, 5], *, [7, 8, 9])`

remove the
marker

`([0, 1, 2, 3, 4, 5], [7, 8, 9])`

Differentiating Algebraic Types

`[0, 1, 2, 3, 4, 5, *, 7, 8, 9]`

split at the
first marker

`([0, 1, 2, 3, 4, 5], *, [7, 8, 9])`

remove the
marker

`([0, 1, 2, 3, 4, 5], [7, 8, 9])`

if there was
a value at
the marker...

`([0, 1, 2, 3, 4, 5], 6, [7, 8, 9])`

Differentiating Algebraic Types

maintain left in
reverse order

```
([5, 4, 3, 2, 1, 0], 6, [7, 8, 9])
```

```
split :: [Either a h] -> Either [a] ([a], h, [Either a h])
```

```
split [] = Left []
```

```
split (Right h : xs) = Right ([], h, xs)
```

```
split (Left x : xs) =
```

```
  case split xs of
```

```
    Left as -> Left (x : as)
```

```
    Right (ps, h, rest) -> Right (x : ps, h, rest)
```

$\Theta(1)$

Differentiating Algebraic Types

maintain left in
reverse order

```
([5, 4, 3, 2, 1, 0], 6, [7, 8, 9])
```

```
zipperAt :: a -> ([a], [a]) -> ([a], a, [a])
```

```
zipperAt x (before, after) = (before, x, after)
```

$$\text{List Zipper} \cong X \times Q(X,0) \cong L(X) \times X \times L(X)$$

The Derivative of a Regular Type is its Type of One-Hole Contexts

Extended Abstract

Conor McBride*

Abstract

Polymorphic regular types are tree-like datatypes generated by polynomial type expressions over a set of free variables and closed under least fixed point. The ‘equality types’ of Core ML can be expressed in this form. Given such a type expression T with x free, this paper shows a way to represent the one-hole contexts for elements of x within elements of T , together with an operation which will plug an element of x into the hole of such a context. One-hole contexts are given as inhabitants of a regular type $\partial_x T$, computed generically from the syntactic structure of T by a mechanism better known as partial differentiation. The relevant notion of containment is shown to be appropriately characterized in terms of derivatives and plugging in. The technology is then exploited to give the one-hole contexts for sub-elements of recursive types in a manner similar to Huet’s ‘zipper’[Hue97].

subtraction, an operation inherently troubled by the need to subtract only smaller things from larger, but as the *inversion* of a kind of *addition* (see figure 1).

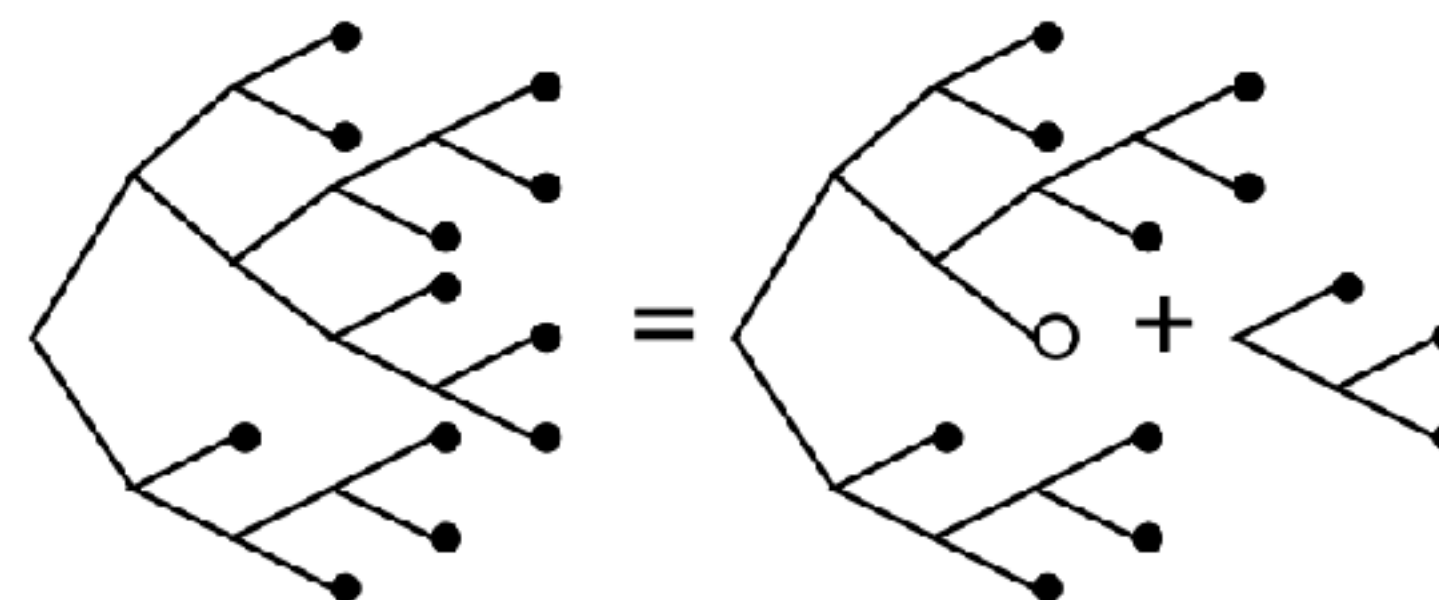


Figure 1: a tree as a one-hole context ‘plus’ a subtree

This paper exhibits a similar technique, defined more formally, which is generic for a large class of tree-like data structures—the ‘regular datatypes’. These are essentially the ‘equality types’ of core ML, presented as polynomial type expressions closed under least fixed point. In particular, I will define and characterize two operations:

1 Introduction

McBride's Theorem (2001)

The derivative of a regular type is its type of one-hole contexts.

McBride's Theorem (2001)

The derivative of a regular type is its type of one-hole contexts.

For every *regular* type, built from $0, 1, X, +, \times$ and recursion, the type ∂F computed by the rules is exactly the type of F -structures with one hole.

McBride's Theorem (2001)

The derivative of a regular type is its type of one-hole contexts.

For every *regular* type, built from $0, 1, X, +, \times$ and recursion, the type ∂F computed by the rules is exactly the type of F -structures with one hole.

For a recursive type $T = F(X, T)$:

$$\partial T = \mu Z. (\partial_X F + \partial_T F \times Z)$$

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

[0, 1, 2, *, 4, 5, 6, *, 8, 9]

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

([0, 1, 2], *, [4, 5, 6], *, [8, 9])

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

([0 , 1 , 2] , [4 , 5 , 6] , [8 , 9])

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

([0 , 1 , 2] , [4 , 5 , 6] , [8 , 9])

The markers are indistinguishable, so there is no factor 2.

What if we differentiate twice?

$$\partial^2 L = \partial(L \times L) = \underbrace{(L \times L) \times L}_{\text{hole on the left}} + \underbrace{L \times (L \times L)}_{\text{hole on the right}} = 2 \times L^3$$

```
data Window a = Window [a] [a] [a]
```



What if we differentiate more?

Exercises:

1. Formally show that the second derivative of a list is a sliding window.
2. Show that storing the sliding window as a two-list queue allows us to compute rolling hash and moving averages in $\Theta(1)$ amortised
3. Split at every marker, not only the first and recover the analogue of the Taylor's theorem in this context:

$$L(X + H) \cong \sum_{k \geq 0} H^k \times L(X)^{k+1}$$



Constructing the Multiway Zipper

```
data Tree a =  
  Tree a [Tree a]
```

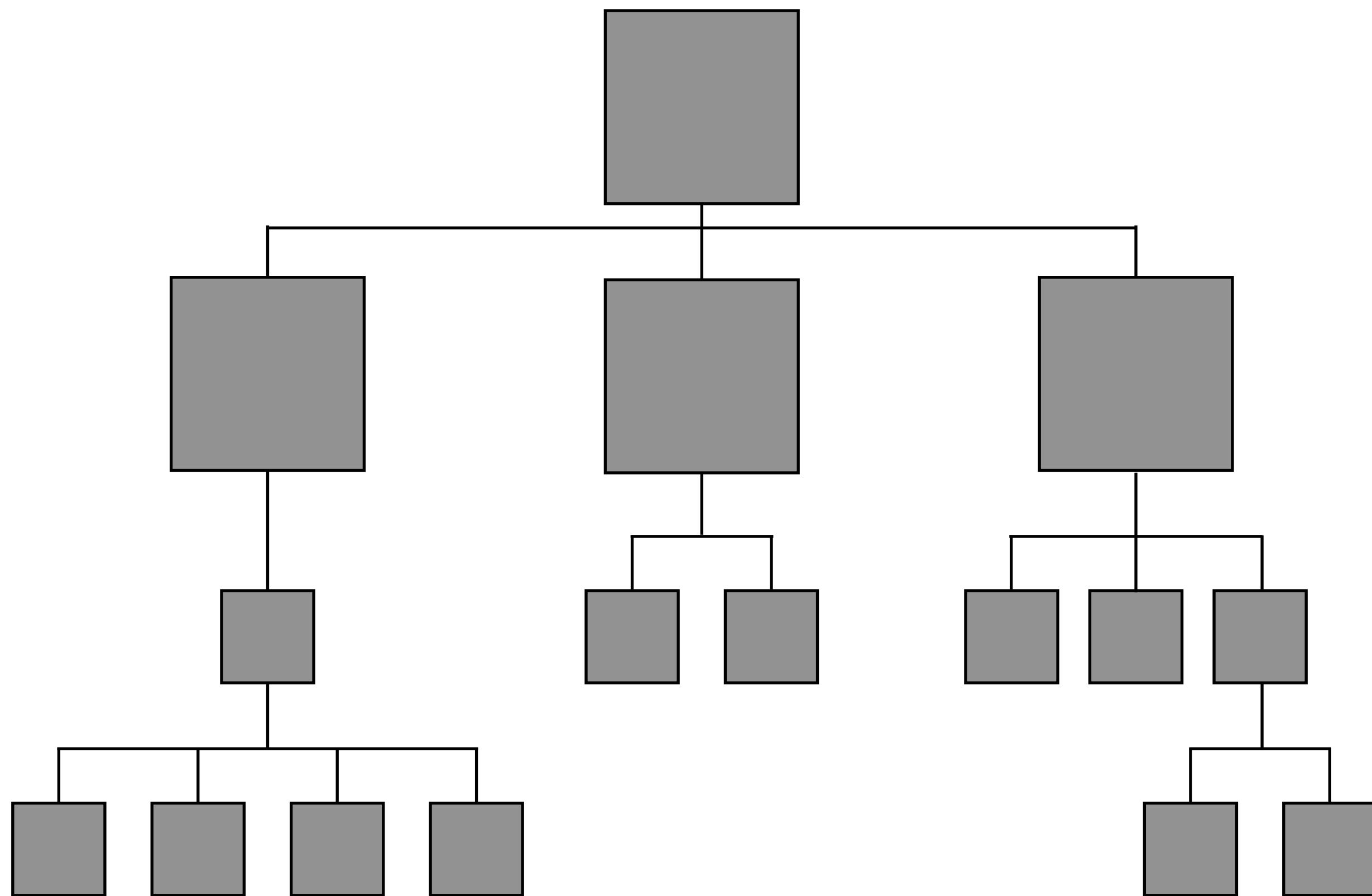
```
leftSiblings  :: [Tree a]
```

```
rightSiblings :: [Tree a]
```

```
focus       :: a
```

```
children     :: [Tree a]
```

```
parent       :: ??
```



Constructing the Multiway Zipper

$$R = X \times L(R)$$

Constructing the Multiway Zipper

$$R = X \times L(R)$$

$$\frac{\partial R}{\partial X} = 1 \times L(R) + X \times \frac{\partial L(R)}{\partial X}$$

Constructing the Multiway Zipper

$$R = X \times L(R)$$

$$\frac{\partial R}{\partial X} = \underbrace{1 \times L(R)}_{\text{children}} + X \times \frac{\partial L(R)}{\partial X}$$

Constructing the Multiway Zipper

$$R = X \times L(R)$$

$$\frac{\partial R}{\partial X} = 1 \times L(R) + X \times \frac{\partial L(R)}{\partial X}$$

$$\frac{\partial}{\partial X}[L(R)] = (\partial L)(R) \times \frac{\partial R}{\partial X} = L(R) \times L(R) \times \frac{\partial R}{\partial X}$$

Constructing the Multiway Zipper

$$R = X \times L(R)$$

$$\frac{\partial R}{\partial X} = L(R) + \left(X \times L(R) \times L(R) \right) \times \frac{\partial R}{\partial X}$$

Constructing the Multiway Zipper

$$Z = L(R) + (X \times L(R) \times L(R)) \times Z$$

Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$



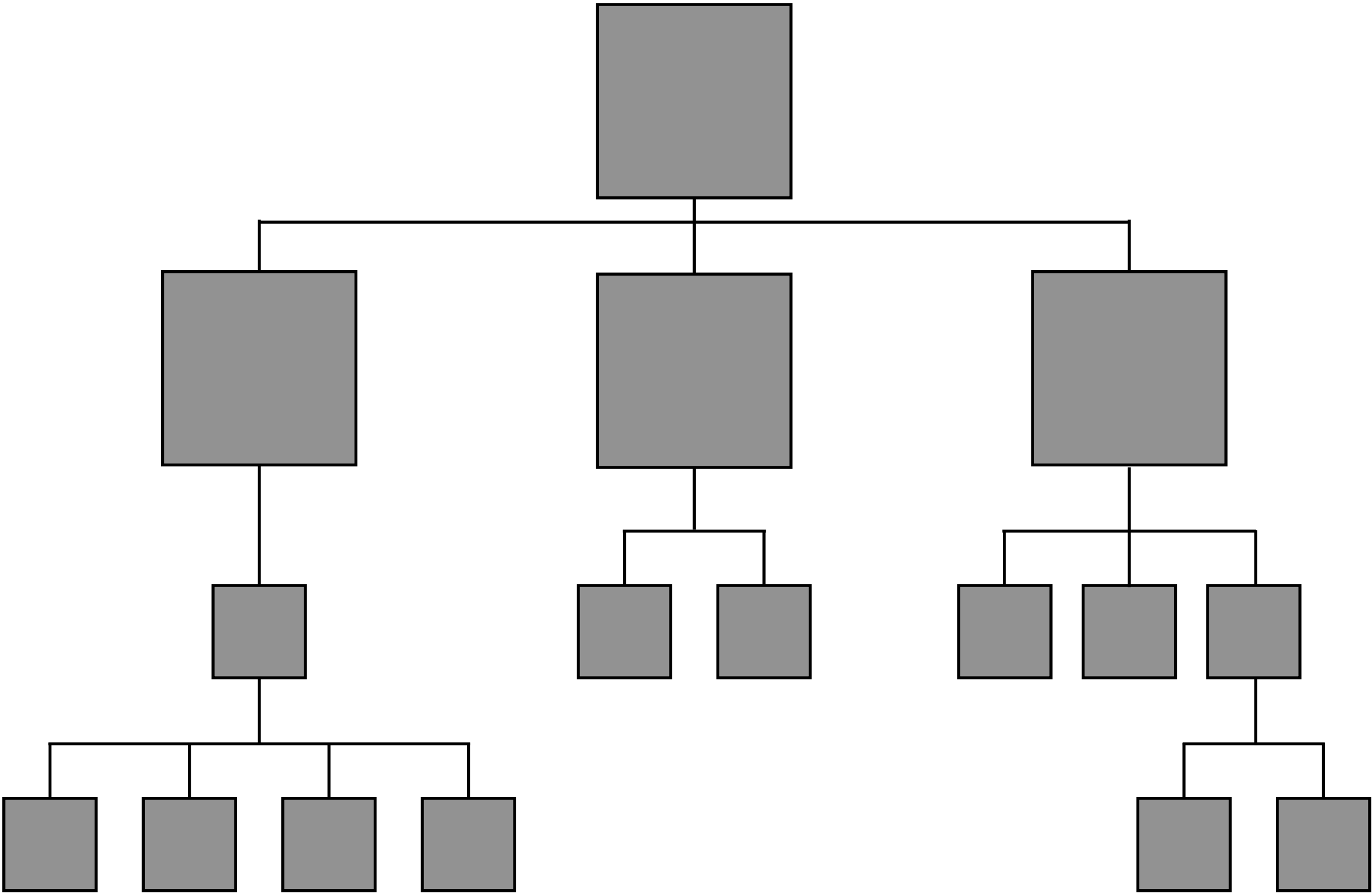
Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

focus :: a

parent ::
[([Tree a], a, [Tree a])]

children :: [Tree a]

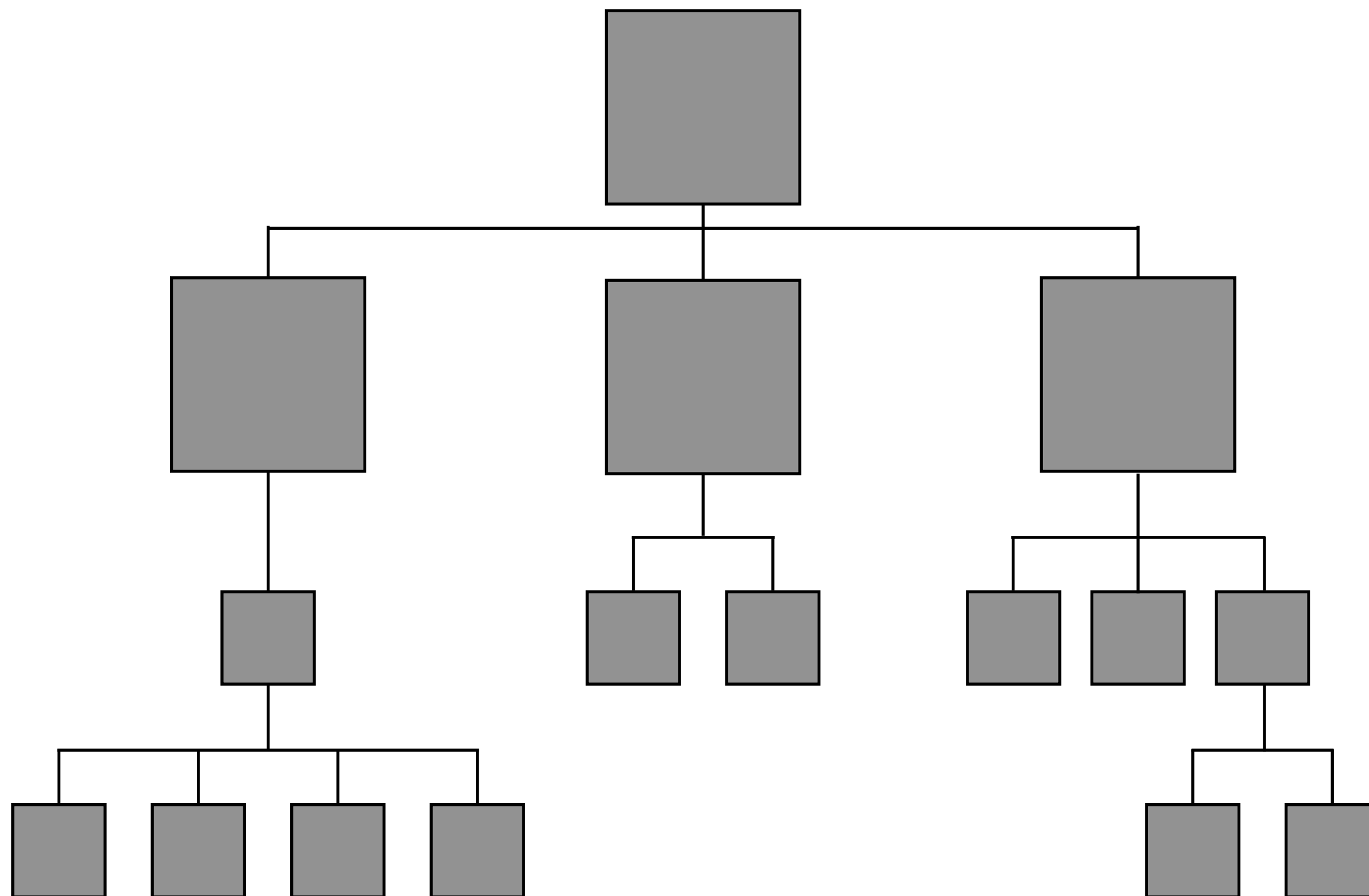




Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

```
data Crumb a =  
  ([Tree a], a, [Tree a])
```



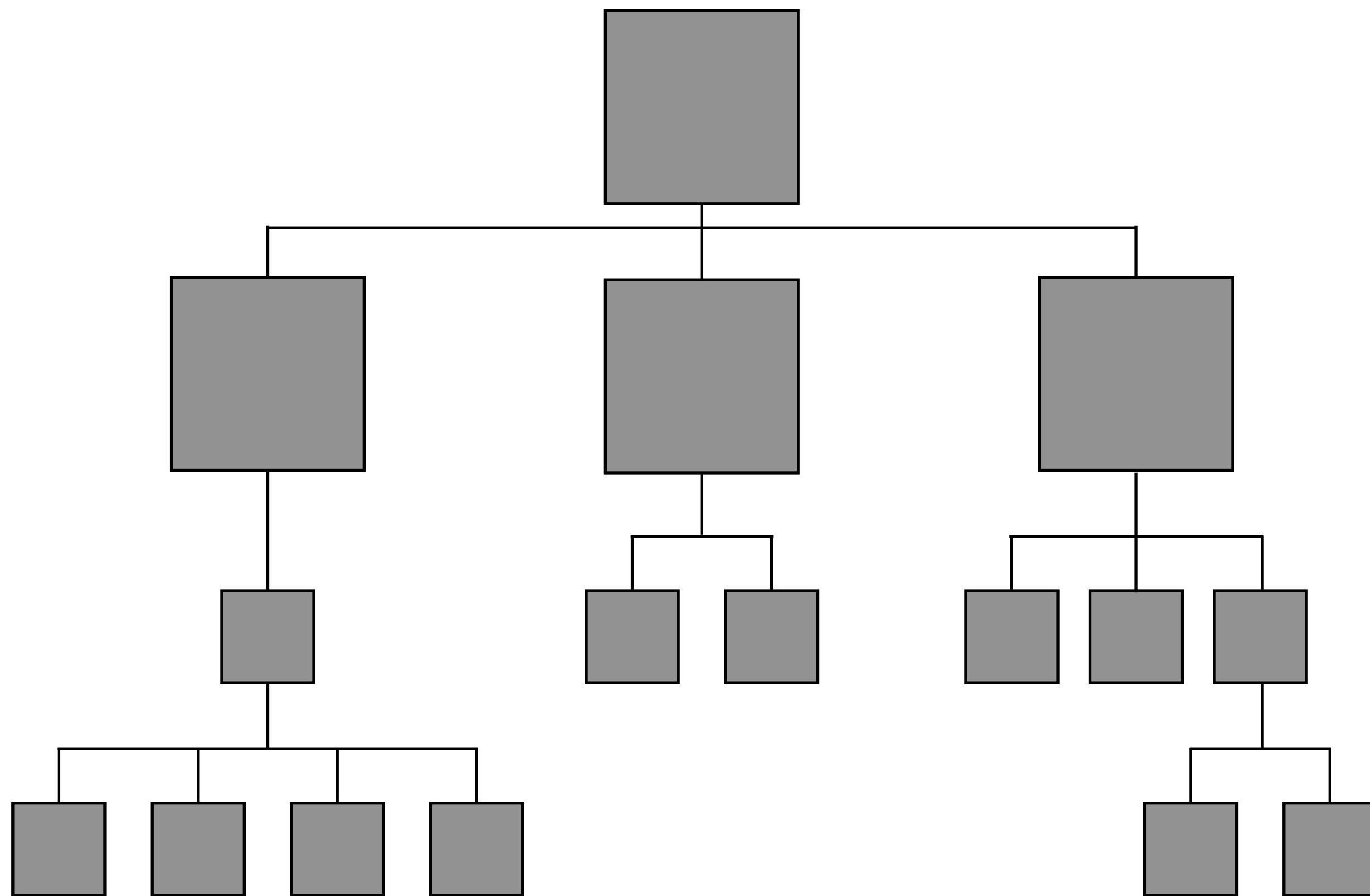


Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

```
data Crumb a =  
  ([Tree a], a, [Tree a])
```

```
data TreeZipper a =  
  TreeZipper  
  a  
  [Crumb a]  
  [Tree a]
```



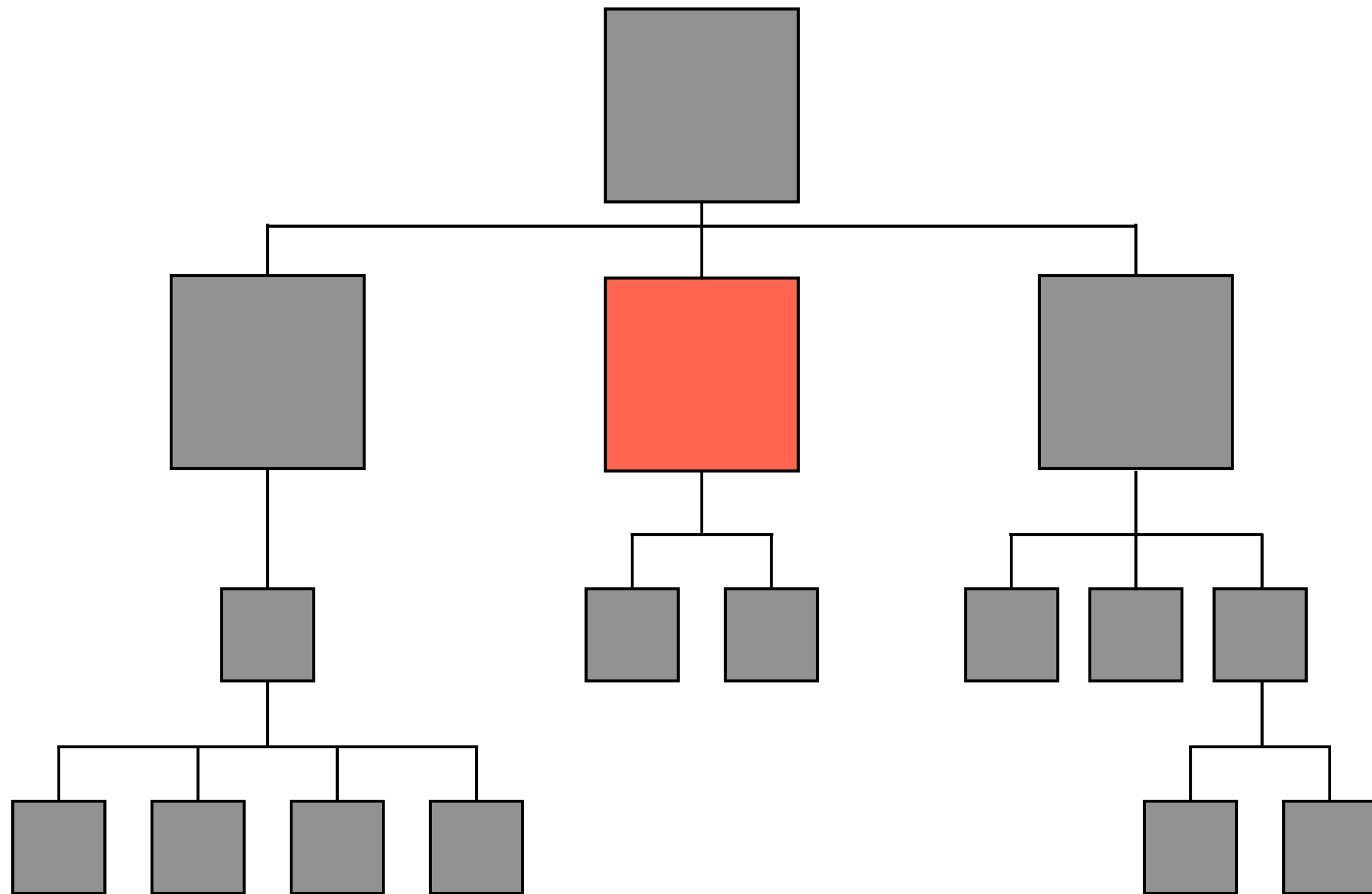


Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

```
data Crumb a =  
  ([Tree a], a, [Tree a])
```

```
data TreeZipper a =  
  TreeZipper  
  a  
  [Crumb a]  
  [Tree a]
```



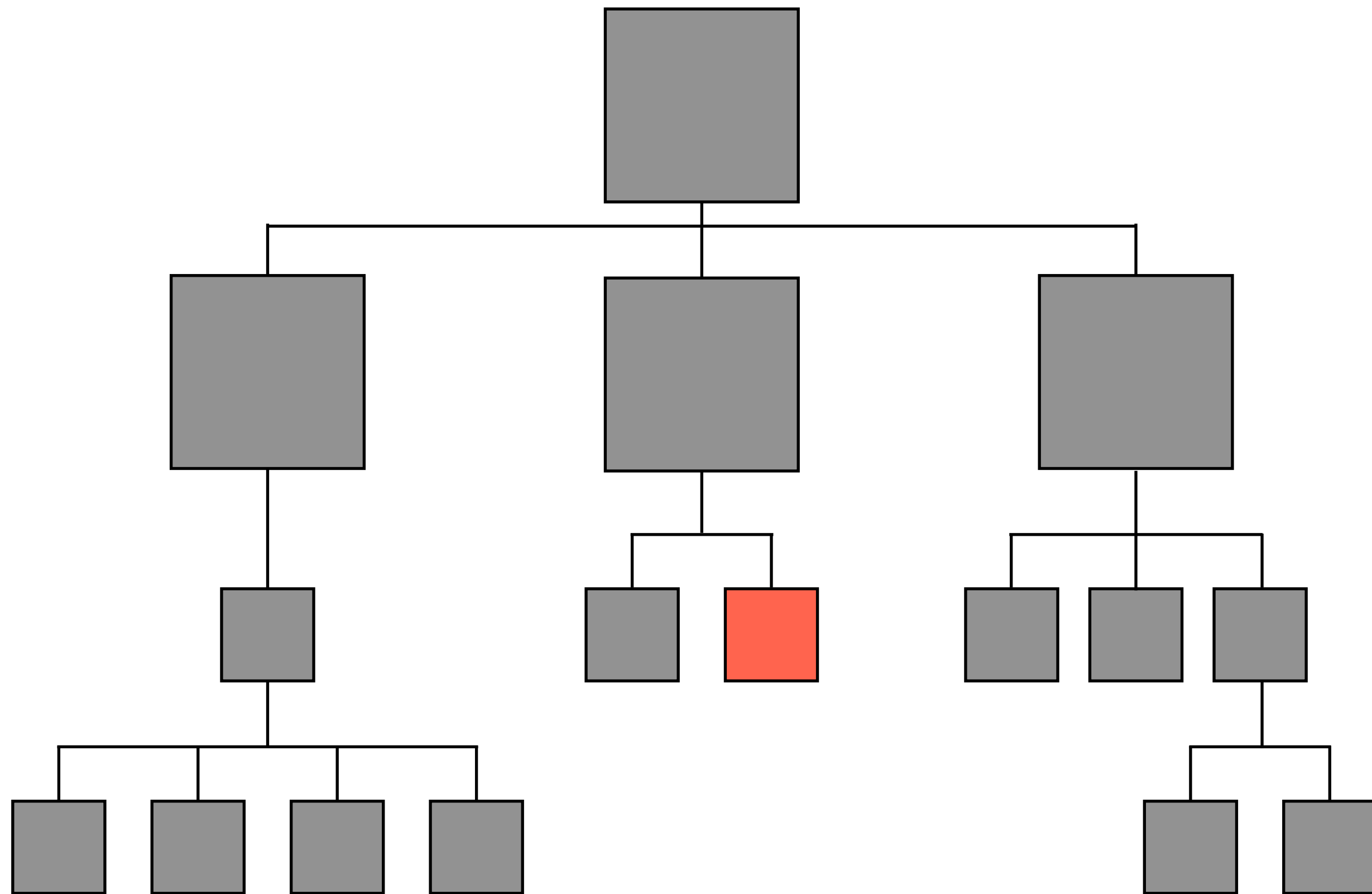


Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

```
data Crumb a =  
  ([Tree a], a, [Tree a])
```

```
data TreeZipper a =  
  TreeZipper  
  a  
  [Crumb a]  
  [Tree a]
```



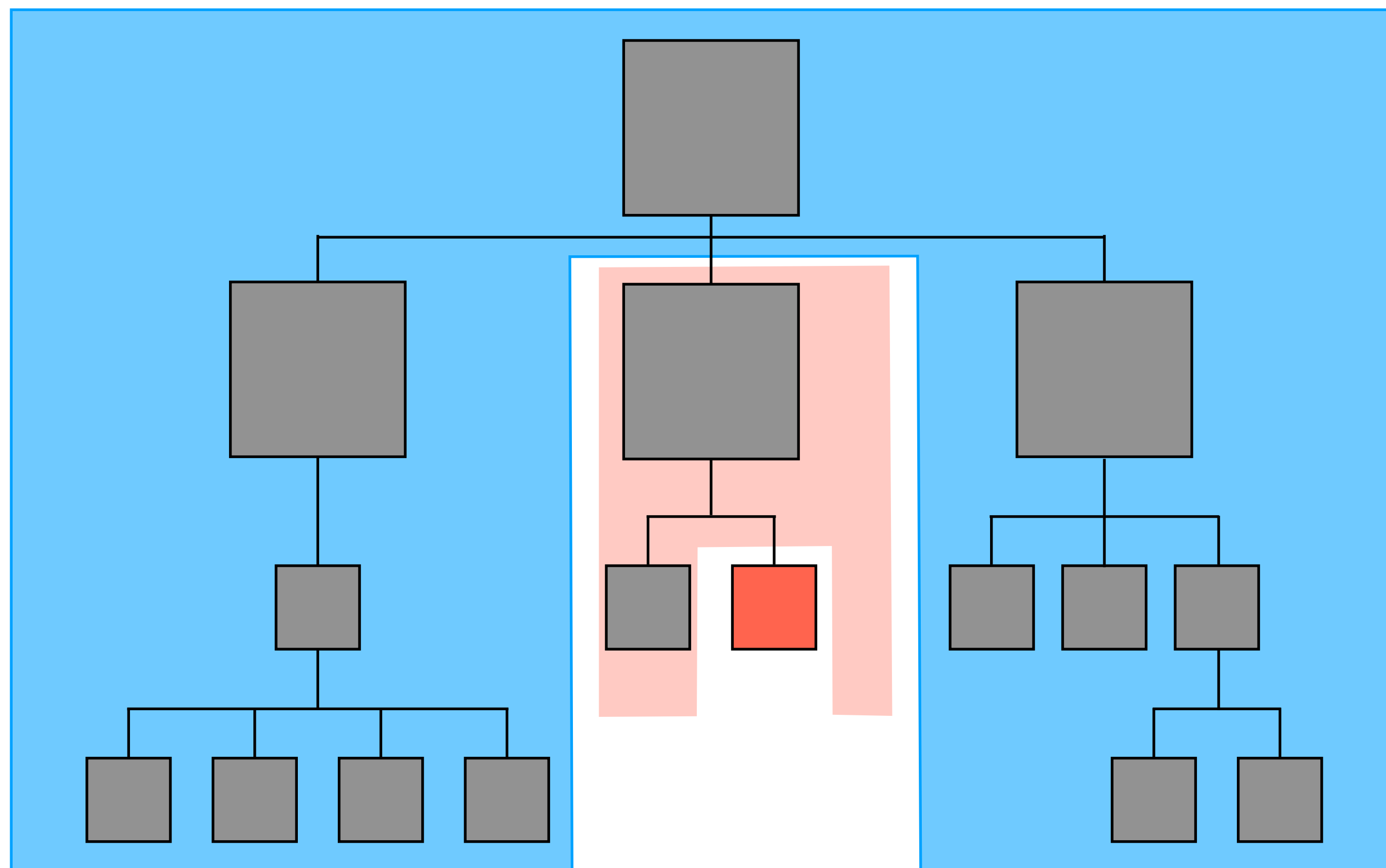


Constructing the Multiway Zipper

$$\text{zipper} = X \times \partial R = X \times L \left(X \times L(R) \times L(R) \right) \times L(R)$$

```
data Crumb a =  
  ([Tree a], a, [Tree a])
```

```
data TreeZipper a =  
  TreeZipper  
  a  
  [Crumb a]  
  [Tree a]
```





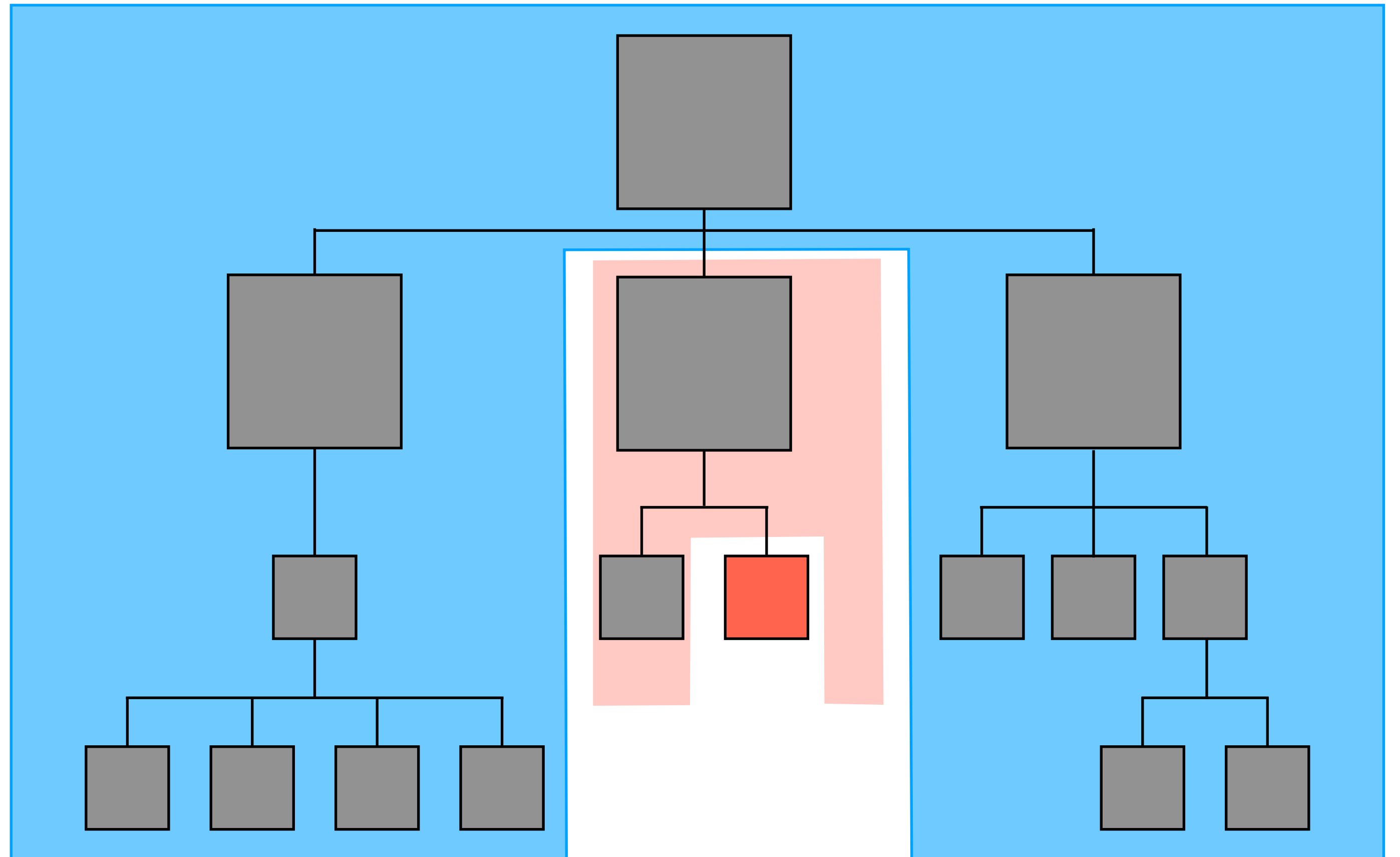
Constructing the Multiway Zipper

A tree zipper is a stack of list zippers.

Exercises:

1. Define right, left, and down such that they take $\Theta(1)$.
2. Define up and analyse its time complexity.
3. Do the same for a binary tree:

$$T = 1 + A \times T^2$$



Zippers as Derivatives

$$\partial \mathbf{0} = \mathbf{0}$$

$$\partial \mathbf{1} = \mathbf{0}$$

$$\mathbf{A} \mapsto \partial A$$

$$\partial(F + G) = \partial F + \partial G$$

$$\partial(F \times G) = (\partial F) \times G + F \times (\partial G)$$

Let $R = \mu T . F(X, T)$. Then:

$$\partial_X R = \mu Z . \left(\partial_X F(X, R) + \partial_T F(X, R) \times Z \right)$$

Zipper as Derivatives

$$\partial \mathbf{0} = \mathbf{0}$$

$$\partial \mathbf{1} = \mathbf{0}$$

$$\mathbf{A} \mapsto \partial A$$

$$\partial(F + G) = \partial F + \partial G$$

$$\partial(F \times G) = (\partial F) \times G + F \times (\partial G)$$

Let $R = \mu T . F(X, T)$. Then:

$$\partial_X R = L(\partial_T F(X, R)) \times \partial_X F(X, R)$$

Stuck in the Middle

A zipper stands still: the same type on both sides of the hole.

A traversal moves: from work done to work not yet done.

Stuck in the Middle

A zipper stands still: the same type on both sides of the hole.

A traversal moves: from work done to work not yet done.

$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$

Stuck in the Middle

A zipper stands still: the same type on both sides of the hole.

A traversal moves: from work done to work not yet done.

$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$

Both sides have different types.

Clowns to the Left of me, Jokers to the Right (*Pearl*)

Dissecting Data Structures

Conor McBride

University of Nottingham

ctm@cs.nott.ac.uk

Abstract

This paper introduces a small but useful generalisation to the ‘derivative’ operation on datatypes underlying Huet’s notion of ‘zipper’ (Huet 1997; McBride 2001; Abbott et al. 2005b), giving a concrete representation to one-hole contexts in data which is undergoing *transformation*. This operator, ‘dissection’, turns a container-like functor into a bifunctor representing a one-hole context in which elements to the left of the hole are distinguished in type from elements to its right.

I present dissection here as a *generic* program, albeit for polynomial functors only. The notion is certainly applicable more widely, but here I prefer to concentrate on its diverse applications. For a start, map-like operations over the functor and fold-like operations over the recursive data structure it induces can be expressed by tail recursion alone. Further, the derivative is readily recovered from the dissection. Indeed, it is the dissection structure which delivers Huet’s operations for *navigating* zippers.

The original motivation for dissection was to define ‘division’, capturing the notion of *leftmost* hole, canonically distinguishing

1. Introduction

There’s an old *Stealer’s Wheel* song with the memorable chorus:

*‘Clowns to the left of me, jokers to the right,
Here I am, stuck in the middle with you.’*

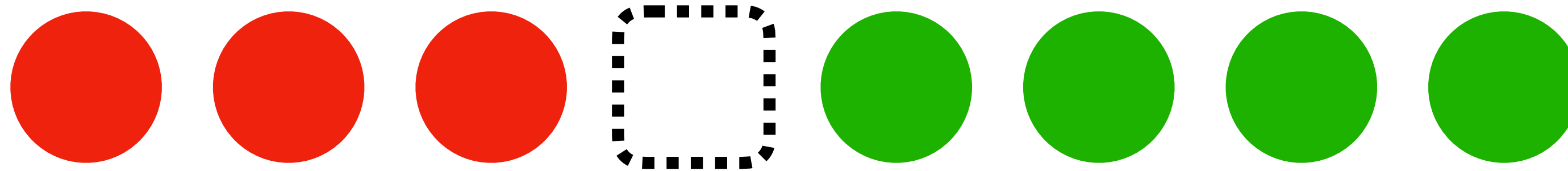
Joe Egan, Gerry Rafferty

In this paper, I examine what it’s like to be stuck in the middle of traversing and transforming a data structure. I’ll show both you and the Glasgow Haskell Compiler how to calculate the datatype of a ‘freezeframe’ in a map- or fold-like operation from the datatype being operated on. That is, I’ll explain how to compute a first-class data representation of the control structure underlying map and fold traversals, via an operator which I call *dissection*. Dissection turns out to generalise both the *derivative* operator underlying Huet’s ‘zippers’ (Huet 1997; McBride 2001) and the notion of *division* used to calculate the non-constant part of a polynomial. Let me take you on a journey into the algebra and differential calculus of datatypes, in search of functionality from structure.

Here’s an example traversal—evaluating a very simple language

Stuck in the Middle

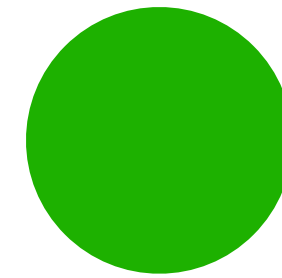
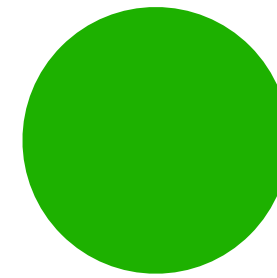
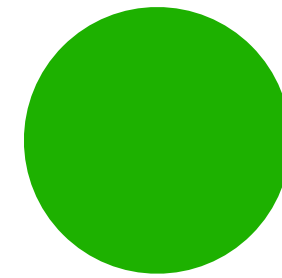
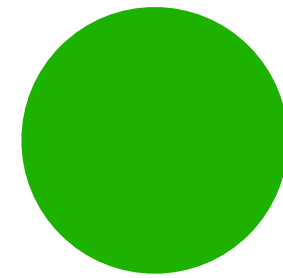
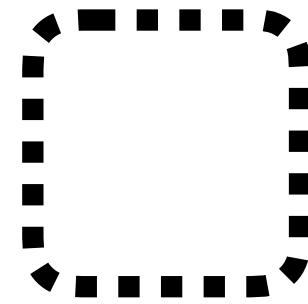
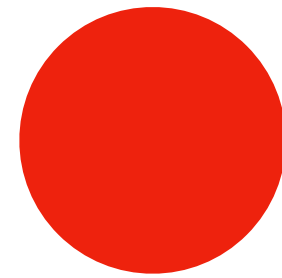
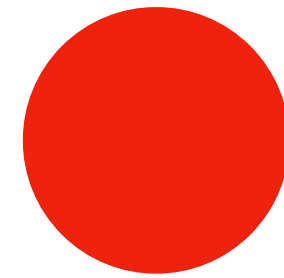
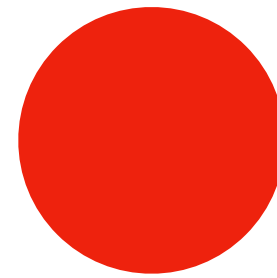
$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$



Stuck in the Middle

$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$

clowns to the
left of me

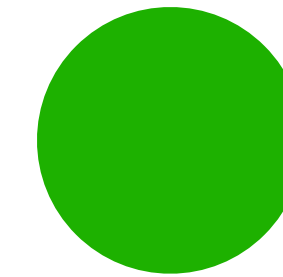
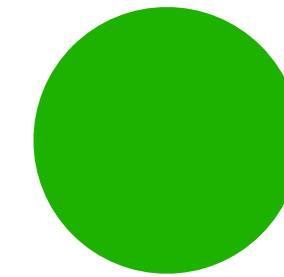
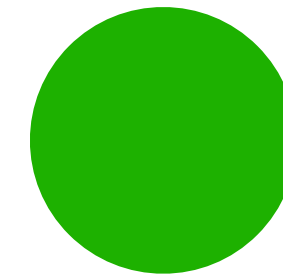
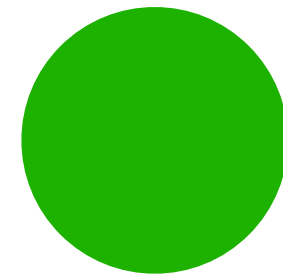
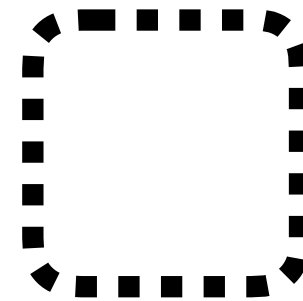
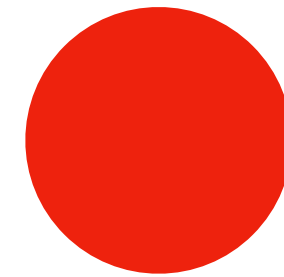
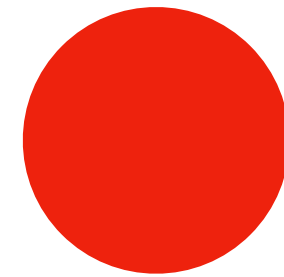
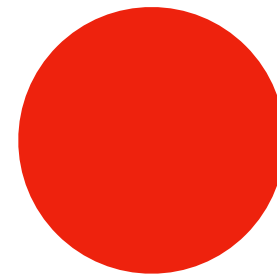


jokers to
the right

Stuck in the Middle

$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$

clowns to the
left of me



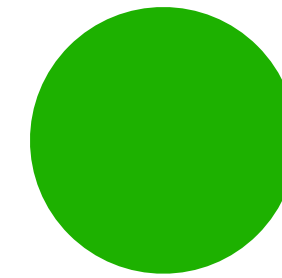
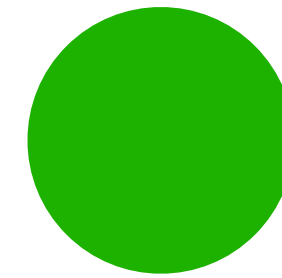
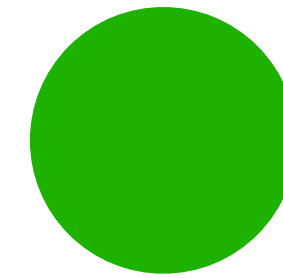
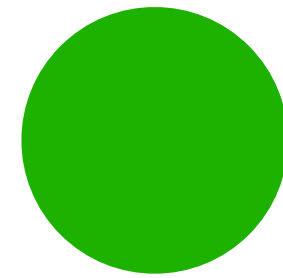
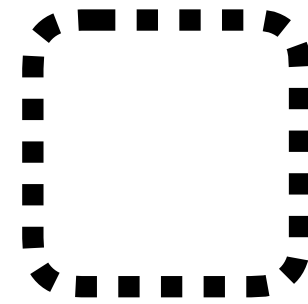
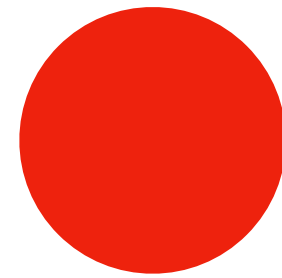
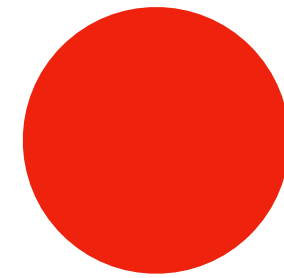
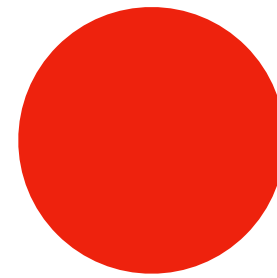
jokers to
the right

([c], [j])

Stuck in the Middle

$$Q(X, H) = \underbrace{L(X)}_{\text{done}} \times \underbrace{L(X + H)}_{\text{not done}}$$

clowns to the
left of me



jokers to
the right

(["5", "4", "3", "2", "1", "0"], 6, [7, 8, 9])

Stuck in the Middle

```
start :: [j] -> Either (j, ([c], [j])) [c]
```

```
start js = next [] js
```

```
step :: ([c], [j]) -> c -> Either (j, ([c], [j])) [c]
```

```
step (cs, js) c = next (c : cs)
```

```
next :: [c] -> [j] -> Either (j, ([c], [j])) [c]
```

```
next cs [] = Right (cs)
```

```
next cs (j : js) = Left (j, (cs, js))
```

Stuck in the Middle

```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
    go (Left (j, d)) = go (step d (f j))
```

```
    go (Right cs) = cs
```

Stuck in the Middle

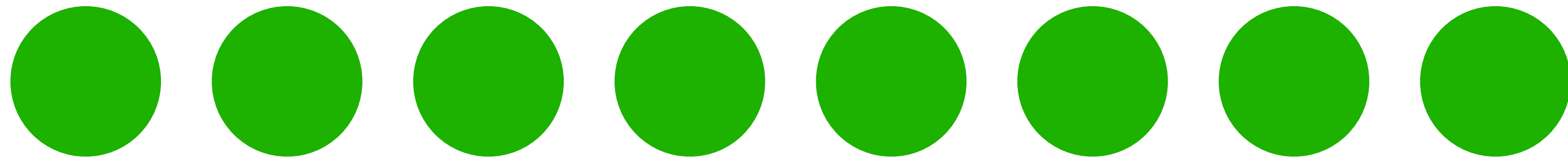
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs) = cs
```



Stuck in the Middle

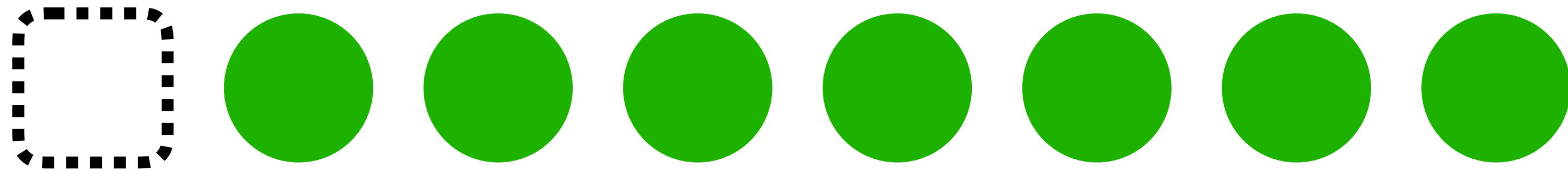
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs) = cs
```



Stuck in the Middle

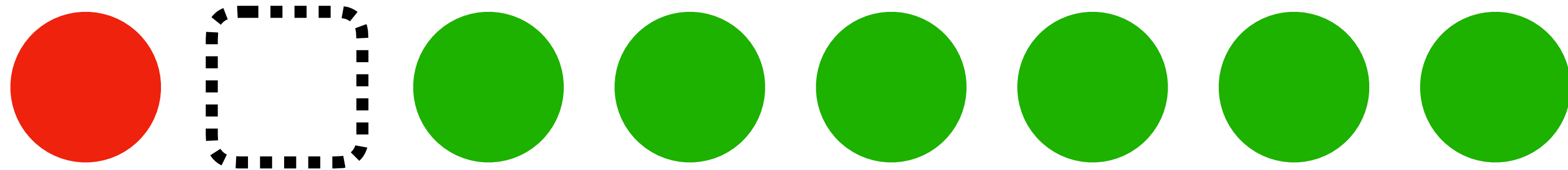
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs)   = cs
```



Stuck in the Middle

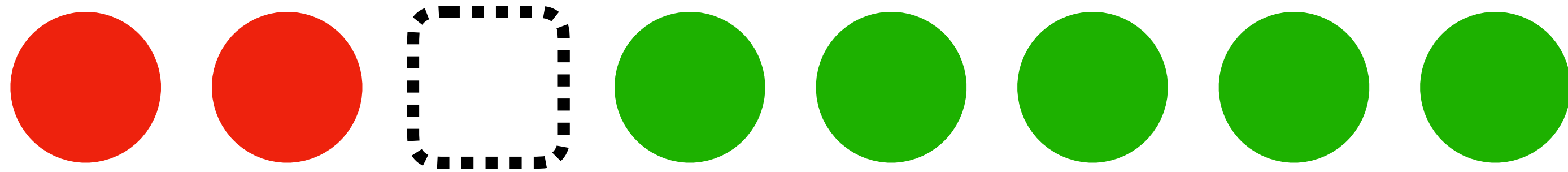
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs)   = cs
```



Stuck in the Middle

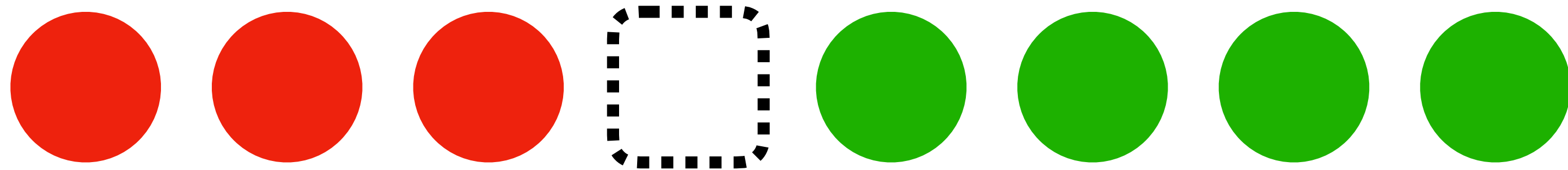
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs)   = cs
```



Stuck in the Middle

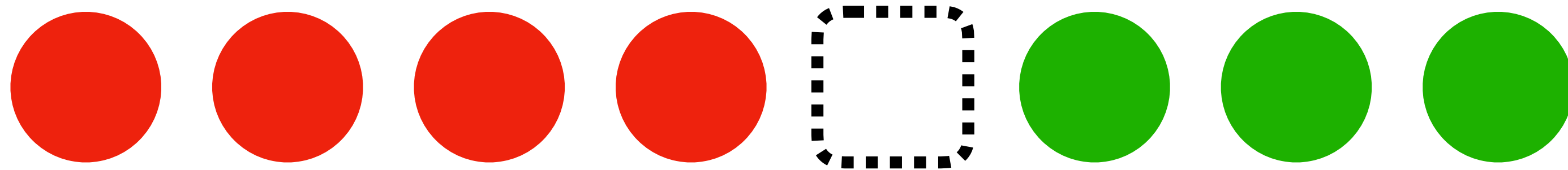
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs)   = cs
```



Stuck in the Middle

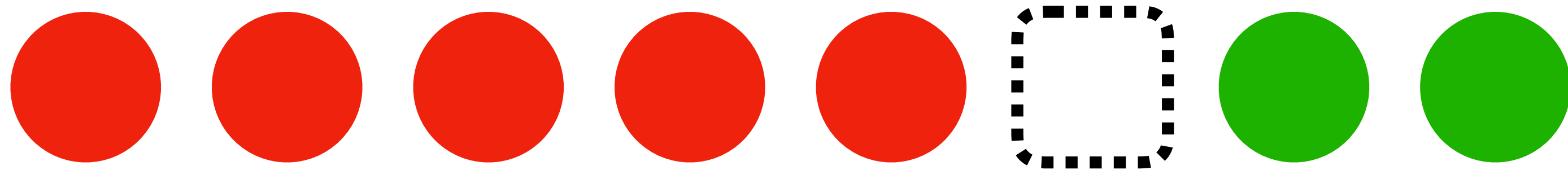
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs)   = cs
```



Stuck in the Middle

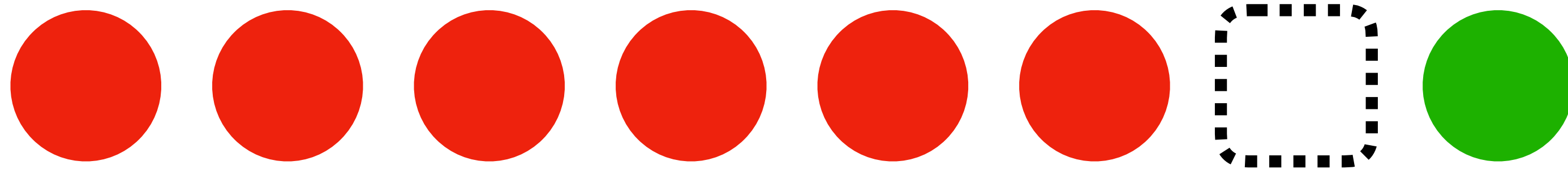
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs) = cs
```



Stuck in the Middle

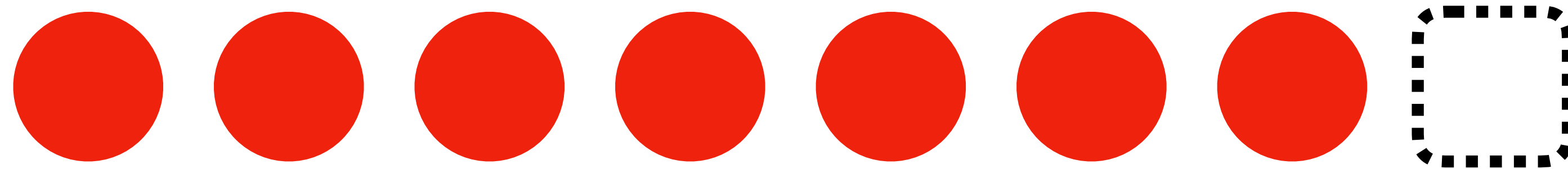
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs) = cs
```



Stuck in the Middle

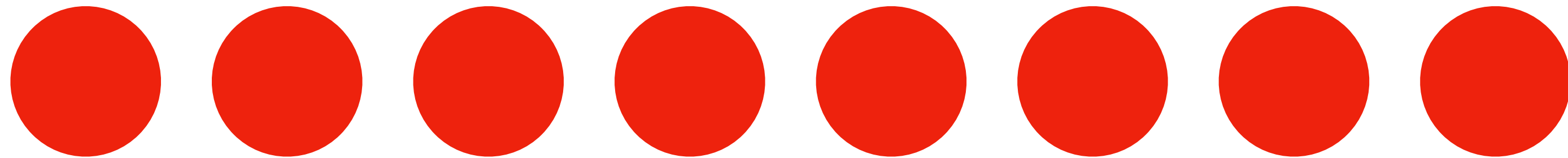
```
mapL :: (j -> c) -> [j] -> [c]
```

```
mapL f js = go (start js)
```

```
where
```

```
  go (Left (j, d)) = go (step d (f j))
```

```
  go (Right cs) = cs
```



Why this matters?

Recursion hides the stack.

```
data Expr = Val Int | Add Expr Expr
```

```
eval :: Expr -> Int
```

```
eval (Val n) = n
```

```
eval (Add l r) = eval l + eval r
```

Why this matters?

Recursion hides the stack.

```
data Expr = Val Int | Add Expr Expr
```

```
eval :: Expr -> Int
```

```
eval (Val n) = n
```

```
eval (Add l r) = eval l + eval r
```

Not a loop: `eval l` must return before `eval r` starts, and `+` waits for both.

Why this matters?

Recursion hides the stack.

```
data Expr = Val Int | Add Expr Expr
```

```
eval :: Expr -> Int
```

```
eval (Val n) = n
```

```
eval (Add l r) = eval l + eval r
```

On a left spine 200,000 deep, with a 1 MB stack: Stack space overflow!

Why this matters?

Recursion hides the stack.

```
data Expr = Val Int | Add Expr Expr
```

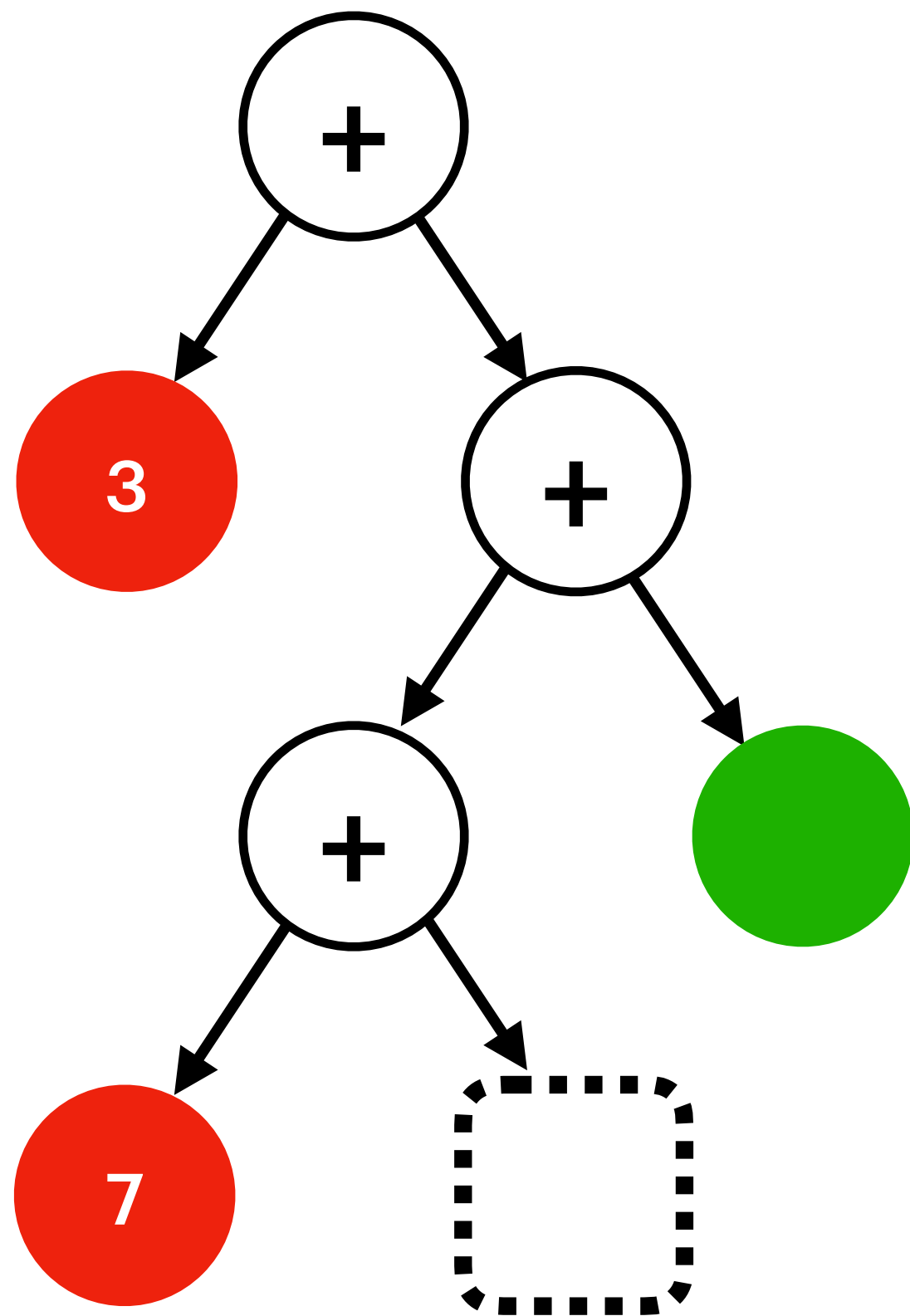
```
eval :: Expr -> Int
```

```
eval (Val n) = n
```

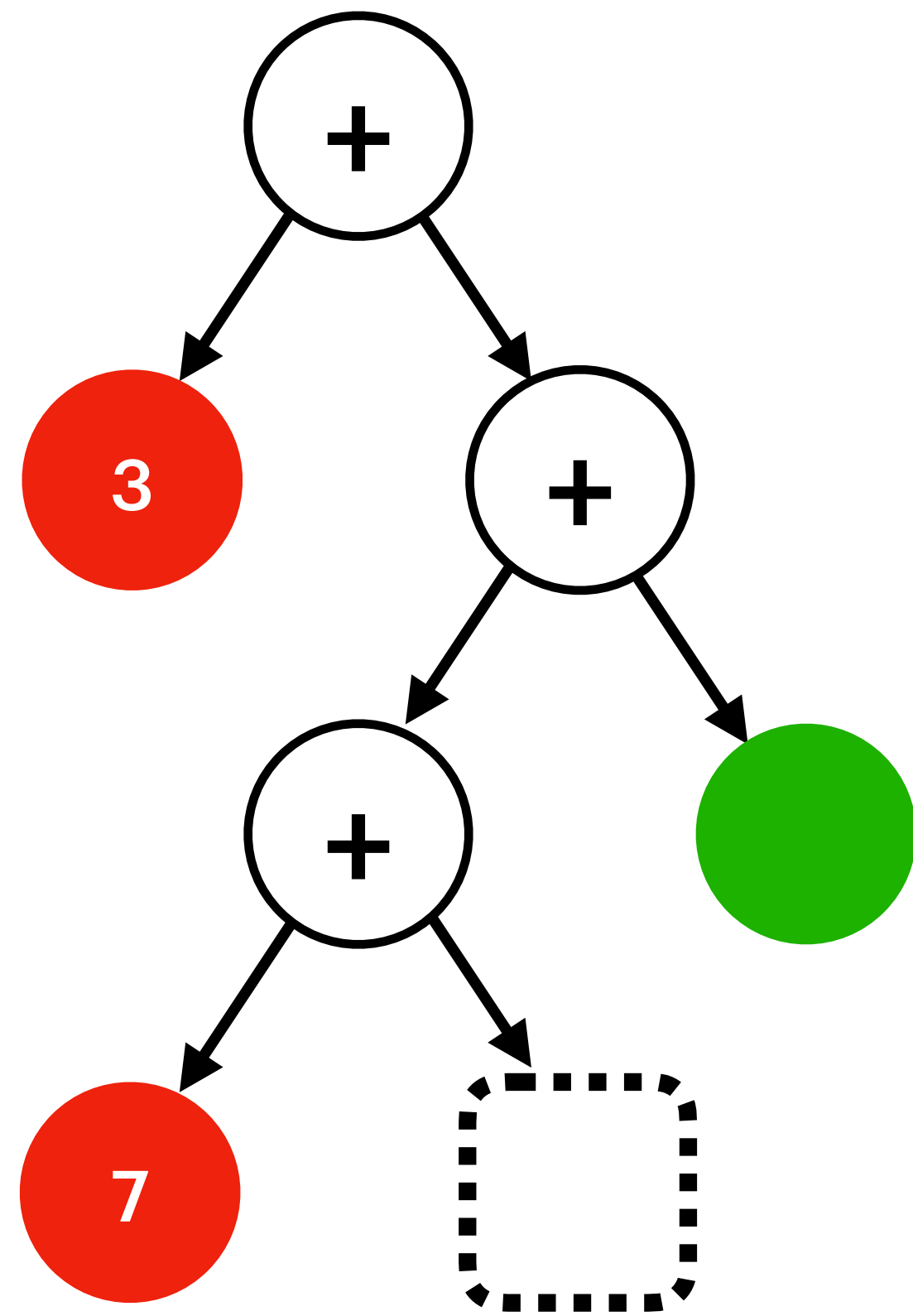
```
eval (Add l r) = eval l + eval r
```

What is the machine remembering?

Why this matters?



Why this matters?



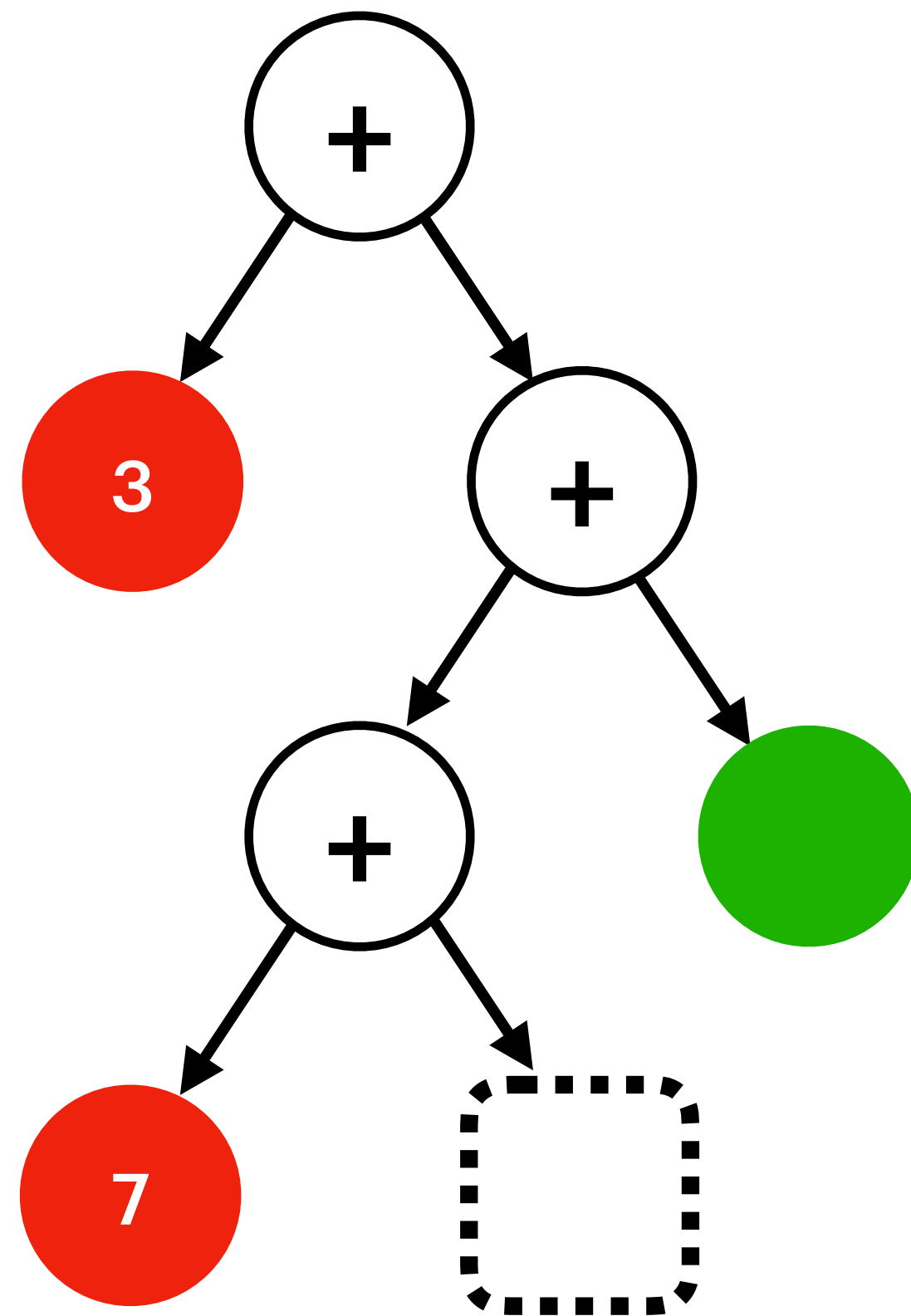
Walk from the root to where we are.

At every `+` on the way:

to the left: clowns, `Val Int`;

to the right: jokers, still `Expr`.

Why this matters?



Walk from the root to where we are.

At every `+` on the way:

to the left: `clowns`, `Val Int`;

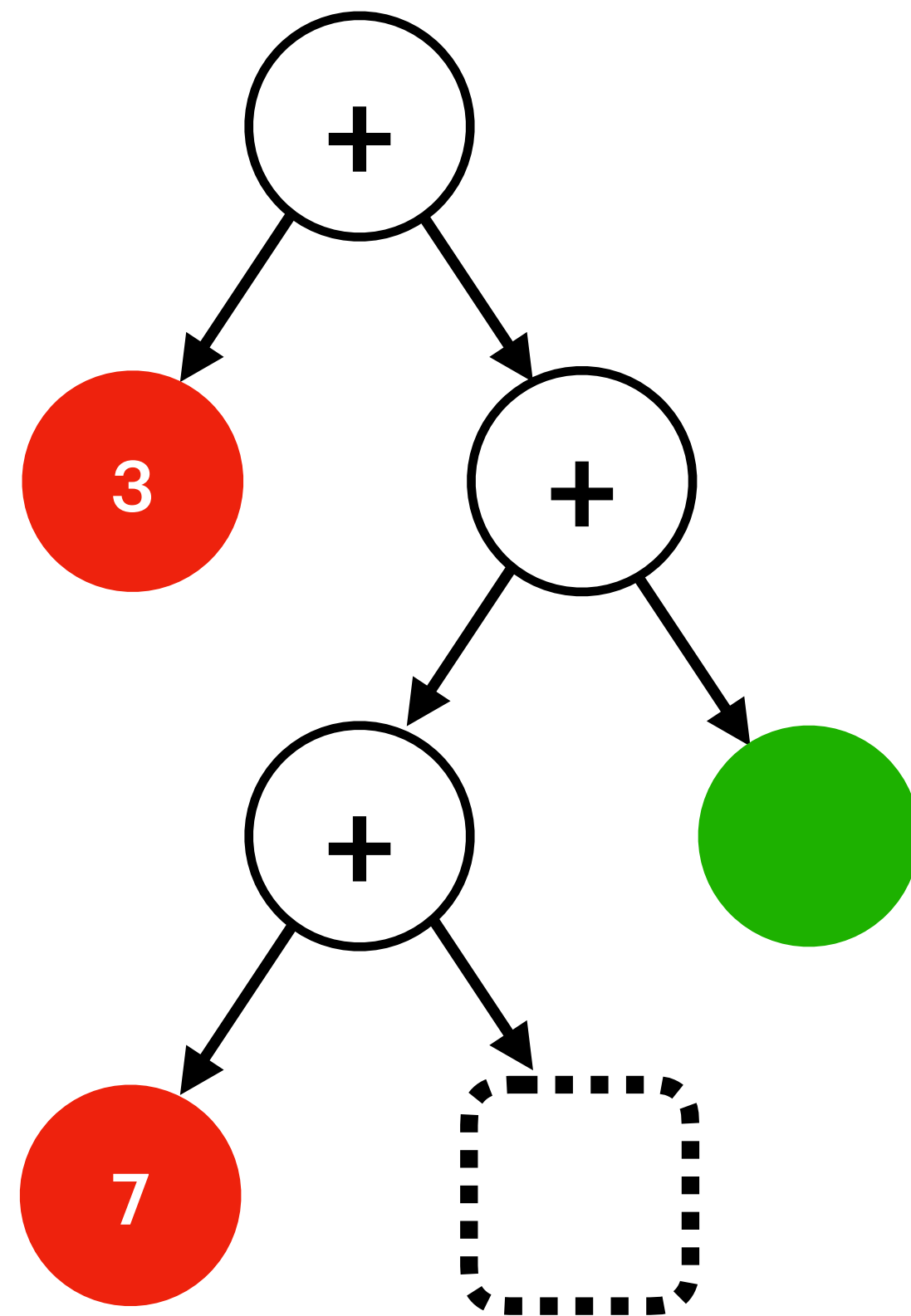
to the right: `jokers`, still `Expr`.

Right 7

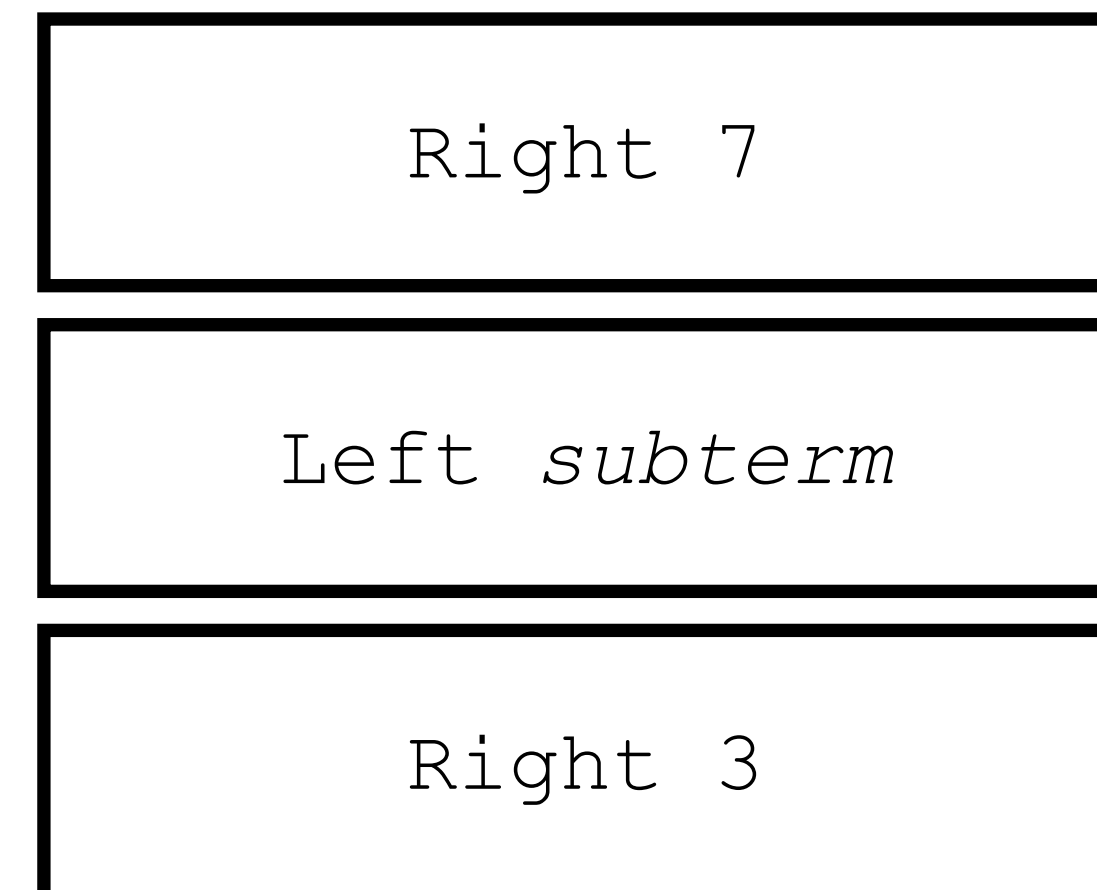
Left *subterm*

Right 3

Why this matters?

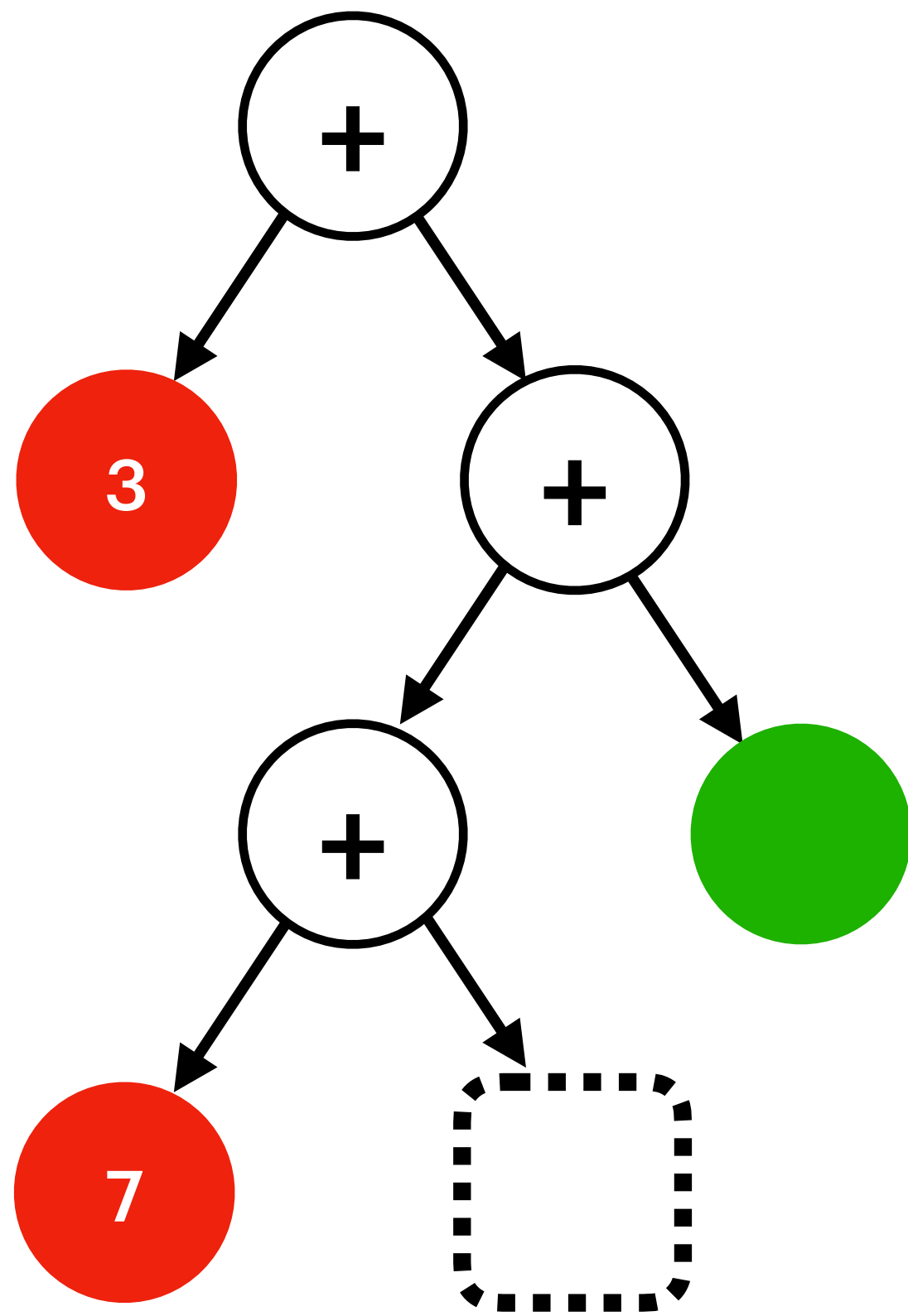


*The call stack is this path:
the list of dissected nodes.*



Why this matters?

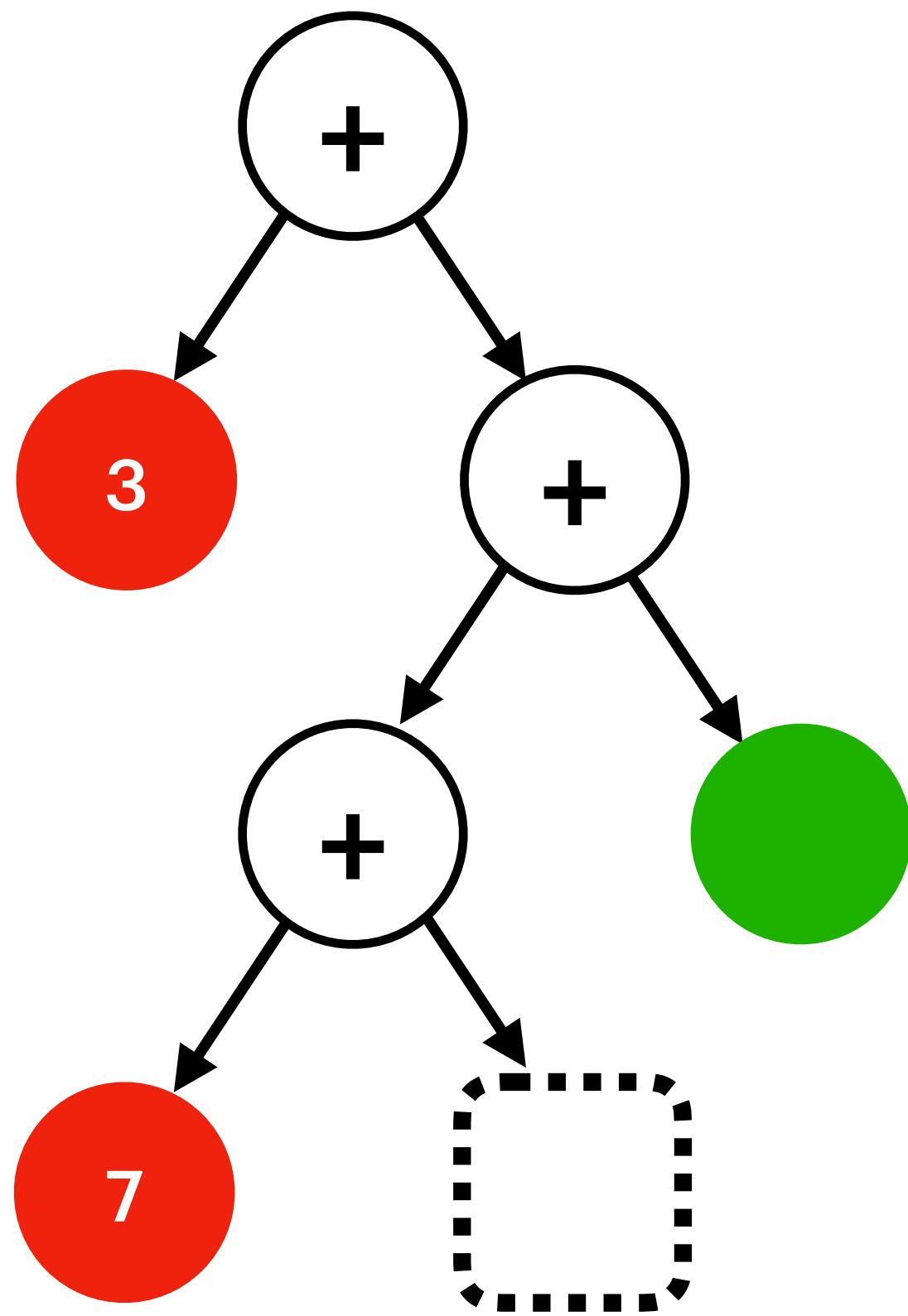
You can go deep: the stack lives in the heap, not in a fixed-size call stack



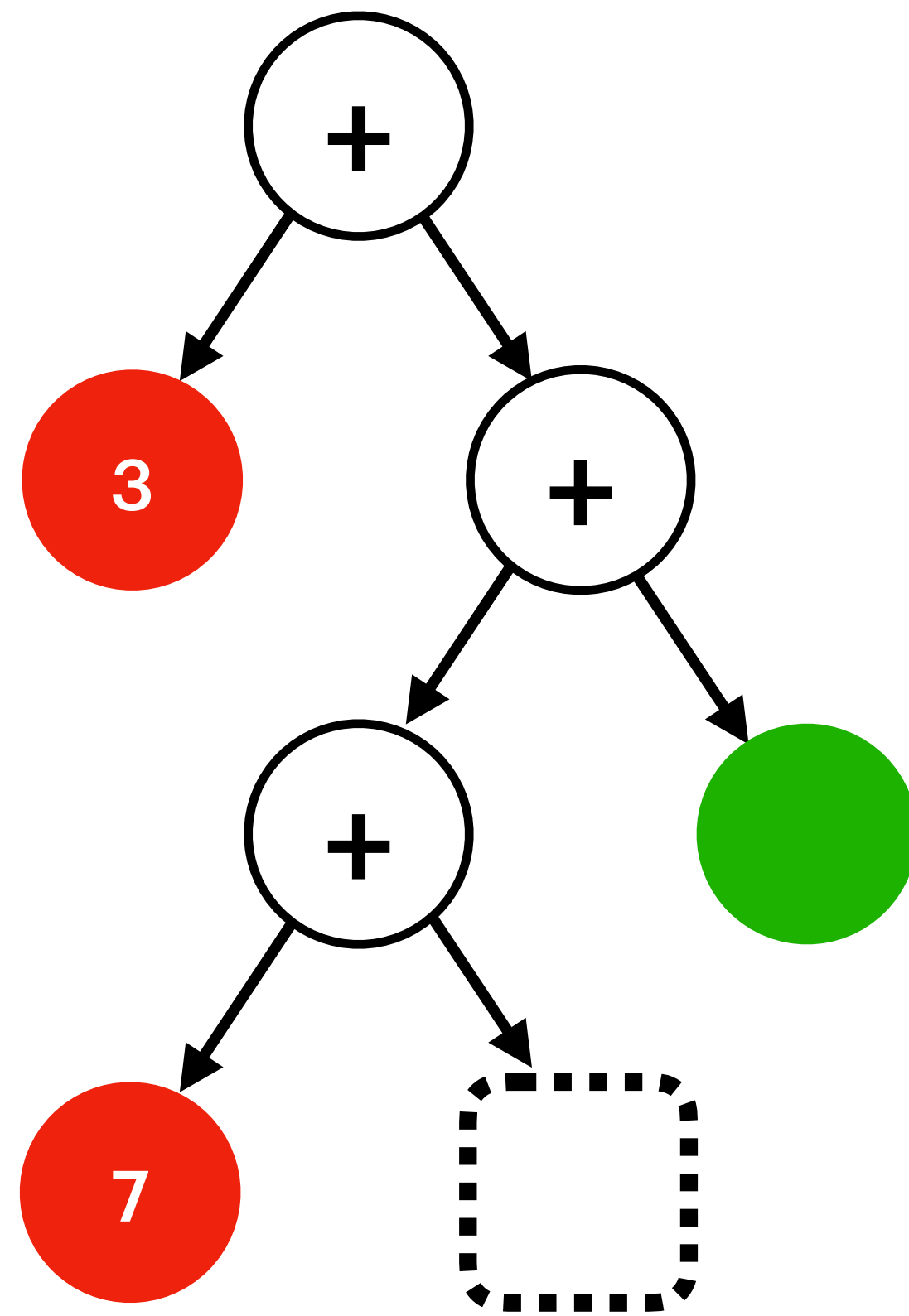
Why this matters?

You can go deep: the stack lives in the heap, not in a fixed-size call stack

You can pause and resume: the whole state of the computation is the stack.



Why this matters?

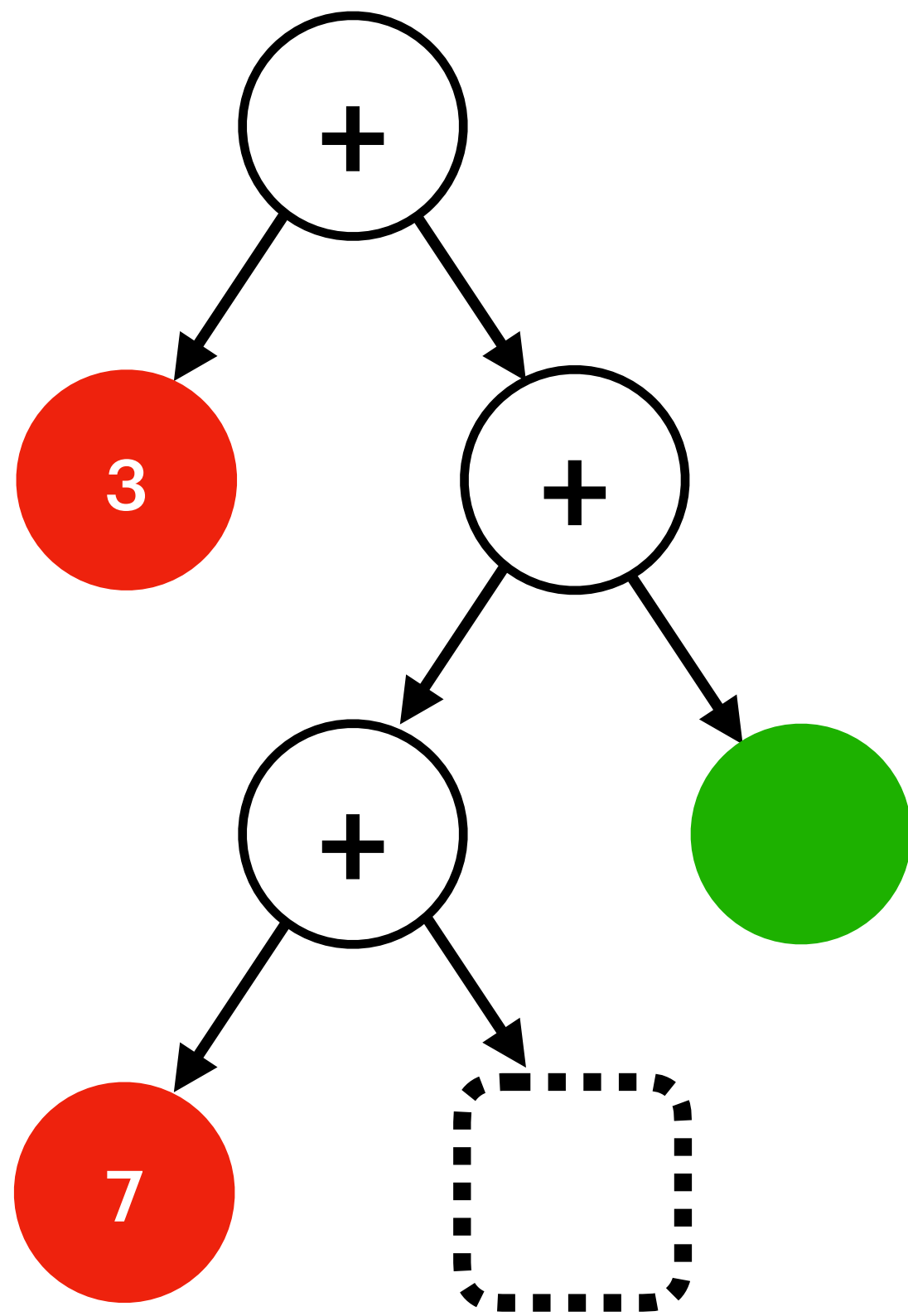


You can go deep: the stack lives in the heap, not in a fixed-size call stack

You can pause and resume: the whole state of the computation is the stack.

How much is done, and what remains, is determined by the focus, clowns, and jokers

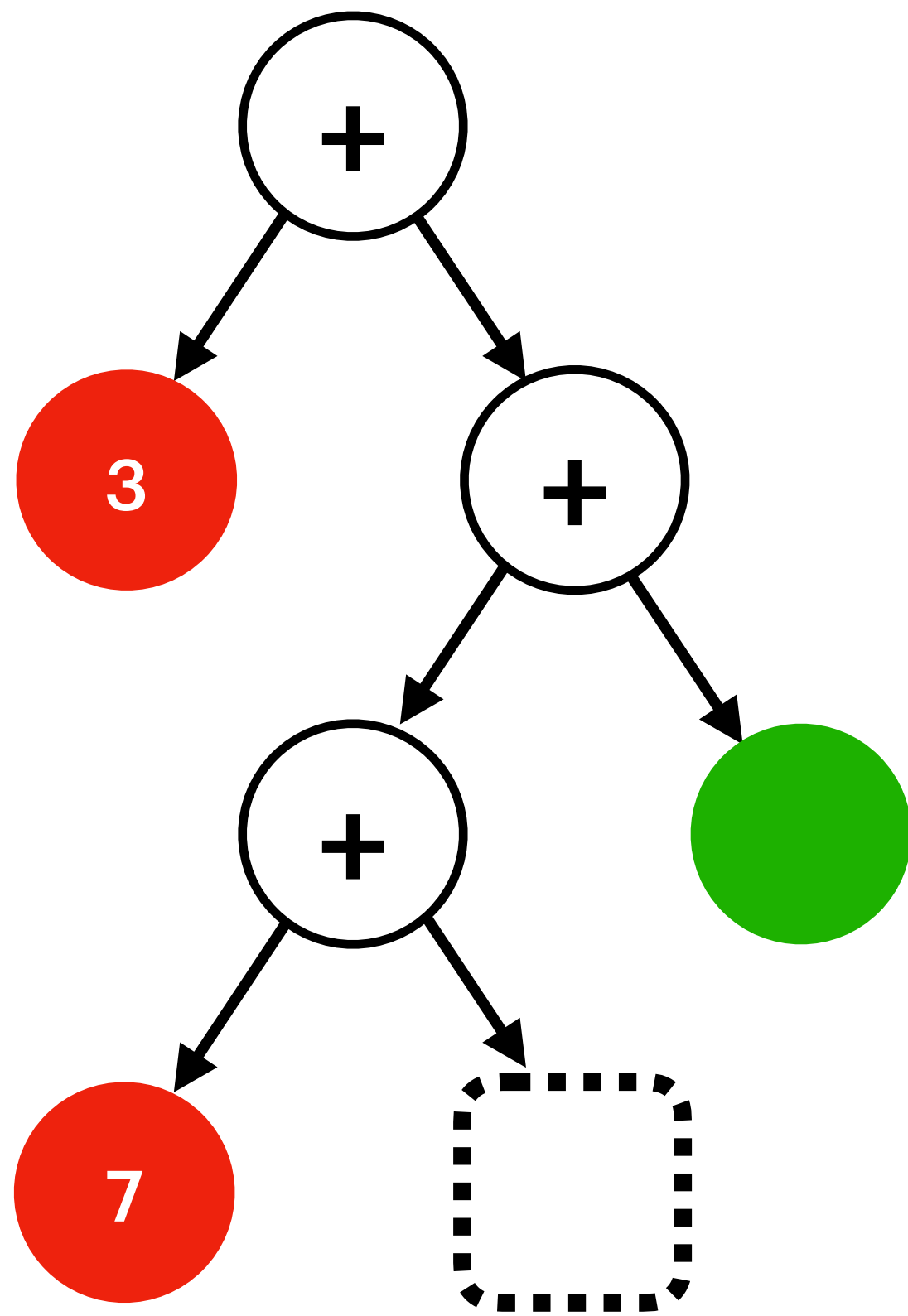
Why this matters?



Here the stack type is calculated from the data type:

$$\underbrace{Int + X^2}_{\text{one layer of Expr}} \xrightarrow{\Delta} \underbrace{J + C}_{\text{one stack frame}} = \text{Either Expr Int}$$

Why this matters?

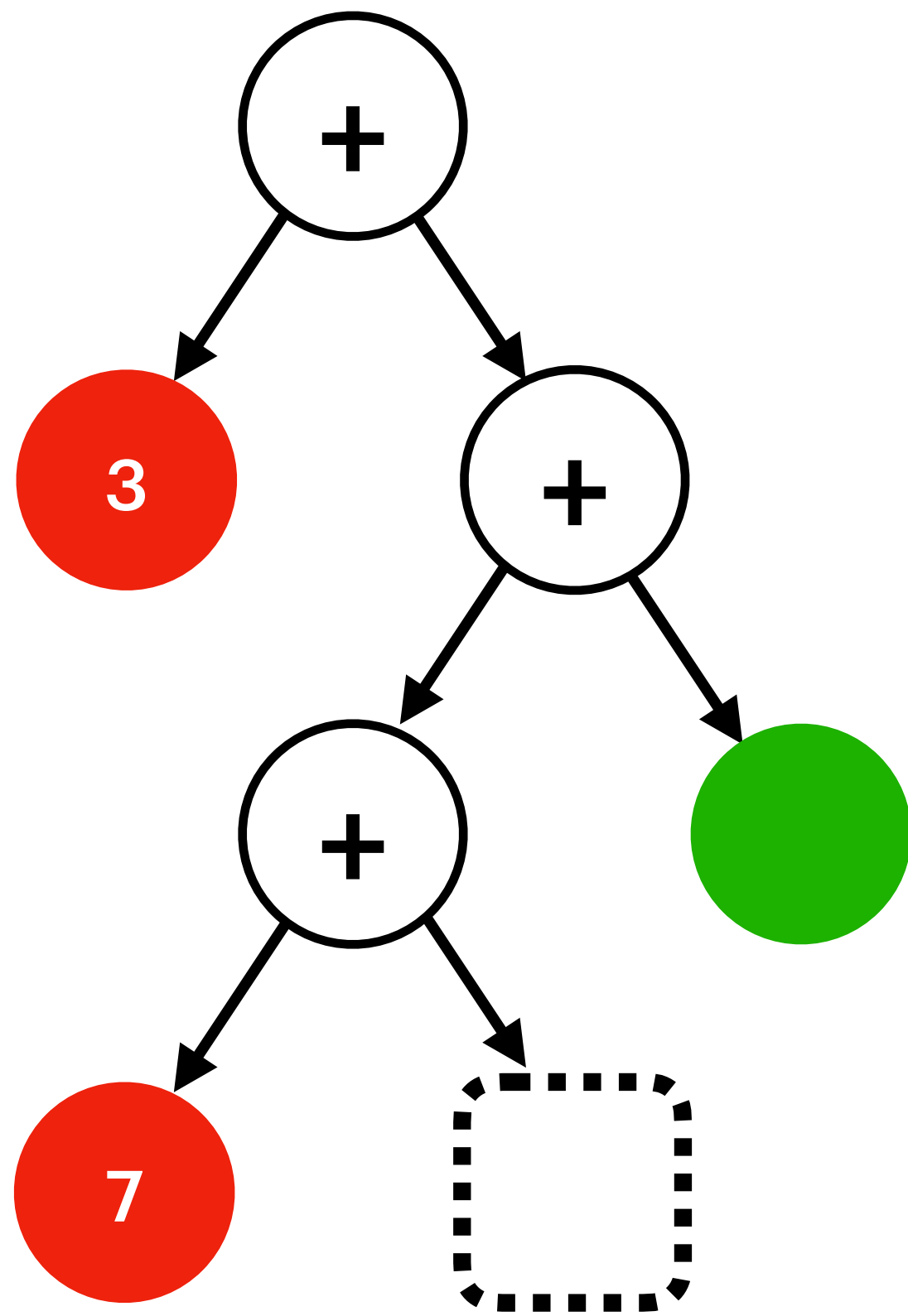


Here the stack type is calculated from the data type:

$$\underbrace{Int + X^2}_{\text{one layer of Expr}} \xrightarrow{\Delta} \underbrace{J + C}_{\text{one stack frame}} = \text{Either Expr Int}$$

Programmers have turned recursion into loops by hand, inventing a stack type each time.

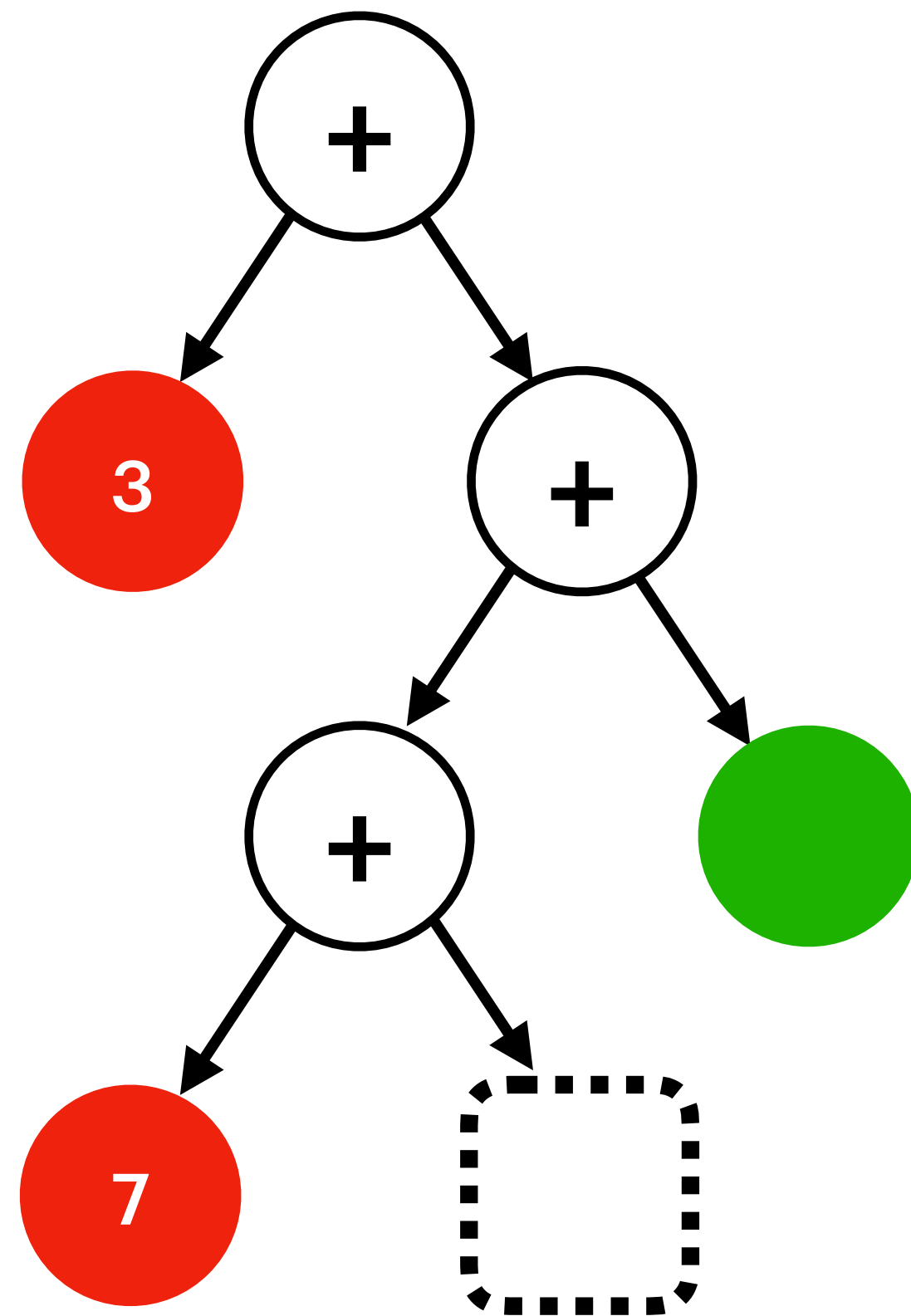
Dissecting Data Structures



A single operation, dissection (making holes), gives us a general theory to understand control as data.

Stacks, continuations and traversals become ordinary values, with types we can compute.

Dissecting Data Structures

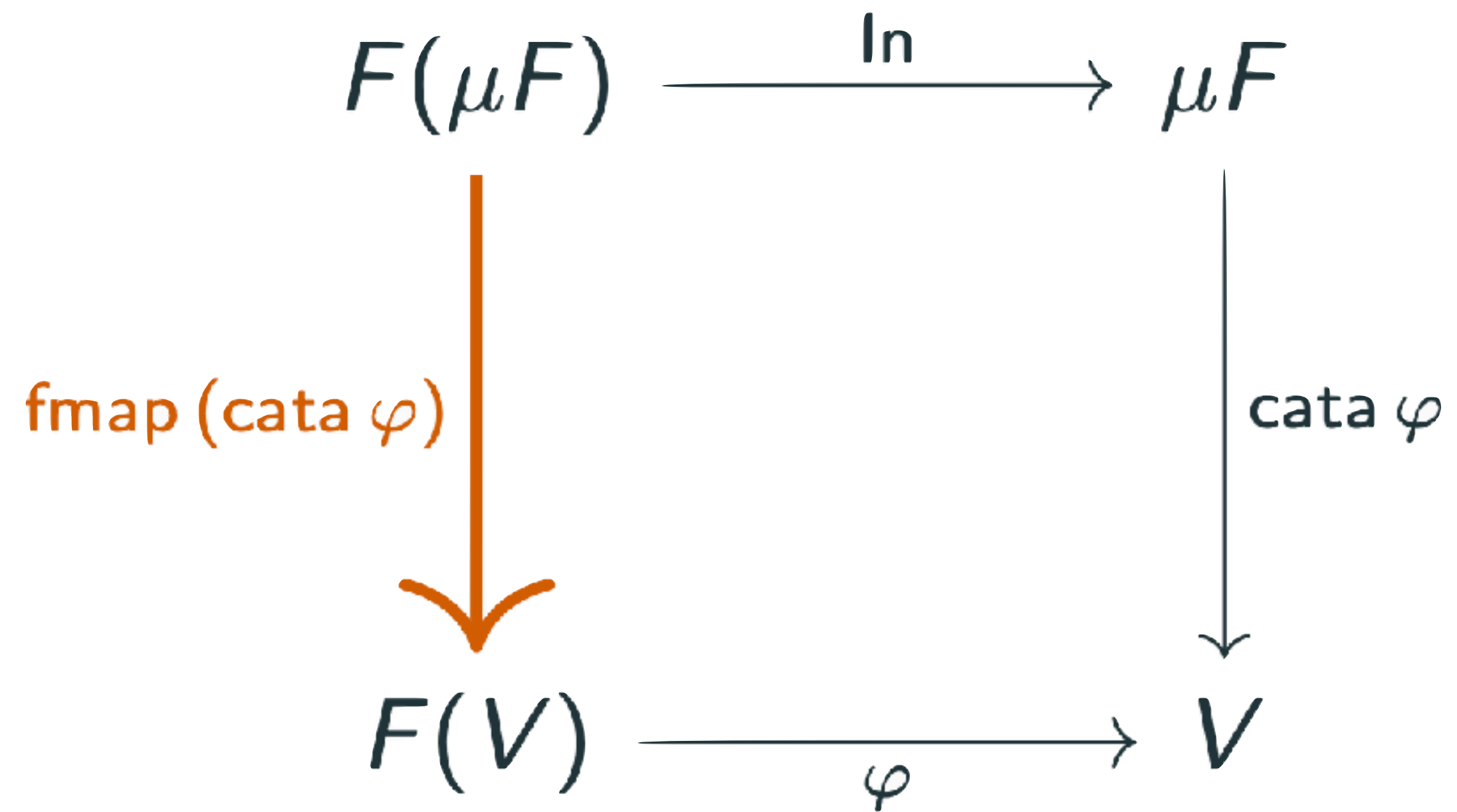


A single operation, dissection (making holes), gives us a general theory to understand control as data.

Stacks, continuations and traversals become ordinary values, with types we can compute.

You said more slides?

Catamorphisms

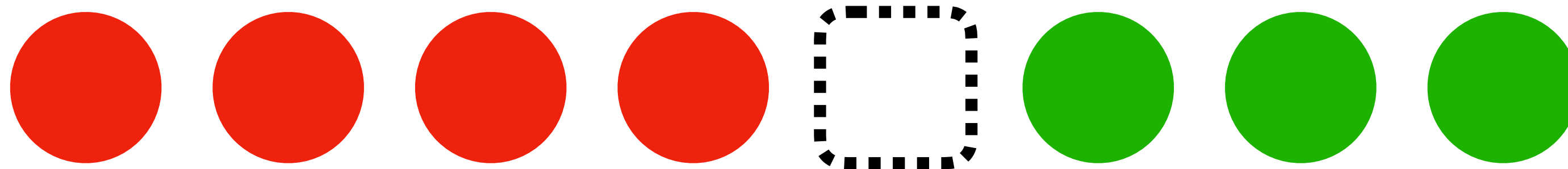


The recursion hides in the left arrow.
`fmap` is a traversal of one layer:

jokers: subtrees, μF

clowns: results, V

$\Delta(V, \mu F)$



Parsing with Zippers (Functional Pearl)

PIERCE DARRAGH, University of Utah, USA

MICHAEL D. ADAMS, University of Michigan, USA

Parsing with Derivatives (PwD) is an elegant approach to parsing context-free grammars (CFGs). It takes the equational theory behind Brzozowski's derivative for regular expressions and augments that theory with laziness, memoization, and fixed points. The result is a simple parser for arbitrary CFGs. Although recent work improved the performance of PwD, it remains inefficient due to the algorithm repeatedly traversing some parts of the grammar.

In this functional pearl, we show how to avoid this inefficiency by suspending the state of the traversal in a zipper. When subsequent derivatives are taken, we can resume the traversal from where we left off without retraversing already traversed parts of the grammar.

However, the original zipper is designed for use with trees, and we want to parse CFGs. CFGs can include shared regions, cycles, and choices between alternates, which makes them incompatible with the traditional tree model for zippers. This paper develops a generalization of zippers to properly handle these additional features. Just as PwD generalized Brzozowski's derivatives from regular expressions to CFGs, we generalize Huet's zippers from trees to CFGs.

The resulting parsing algorithm is concise and efficient: it takes only 31 lines of OCaml code to implement the derivative function but performs 6,500 times faster than the original PwD and 3.24 times faster than the optimized implementation of PwD.

References

G. Huet. *The Zipper*. Journal of Functional Programming 7(5), 1997.

C. McBride. *The Derivative of a Regular Type is its Type of One-Hole Contexts*. Manuscript, 2001.

M. Abbott, T. Altenkirch, N. Ghani, C. McBride. *∂ for Data: Differentiating Data Structures*. Fundamenta Informaticae 65, 2005.

C. McBride. *Clowns to the Left of Me, Jokers to the Right: Dissecting Data Structures*. POPL 2008.

P. Darragh, M. D. Adams. *Parsing with Zippers*. ICFP 2020.

